

École Polytechnique Fédérale de Lausanne

Thermal properties of MOFs from molecular dynamics

Master Thesis

Lucas De Almeida Kloss

October 30, 2026

Acknowledgments

[Add acknowledgments here.]

Contents

Acknowledgments 1

1 Scientific motivation and scope 5

2 Theory 6
- 2.1 Quantum Mechanics . . . 6
- 2.2 Density Functional Theory . . . 7
- 2.3 Statistical Mechanics . . . 8
- 2.4 Molecular dynamics . . . 13
- 2.5 Machine learning and deep learning . . . 15
- 2.6 Machine-learned interatomic potentials . . . 19
- 2.7 Harmonic force constants and automatic differentiation . . . 22

3 Methods 24
- 3.1 Frameworks and simulation systems . . . 24
- 3.2 Structure preparation and initialization . . . 24
- 3.3 Interatomic potentials and implementations . . . 25
- 3.4 Classical NPT simulations and enthalpy analysis . . . 25
- 3.5 Harmonic references and heat-capacity assembly . . . 26
- 3.6 Uncertainty quantification . . . 27

4 Results 29
- 4.1 Available calculations and status . . . 29
- 4.2 Thermodynamic sampling and equilibration . . . 29
- 4.3 Cell response, framework motion, and methane mobility . . . 30
- 4.4 Classical, harmonic, and hybrid heat capacities . . . 31
- 4.5 Hessian stability . . . 33
- 4.6 Uncertainty bands . . . 33

5 Recommended production and analysis protocol 35

6 Implementation, outputs, and reproducibility 36

7 Limitations and interpretation 37

8 Conclusions 38

A Transformers and attention 39
- A.1 Tokenization and embedding . . . 39
- A.2 Attention mechanism . . . 40
- A.3 Multi-head attention . . . 41
- A.4 Feedforward operations, residual paths, and normalization . . . 41

B Sparse automatic differentiation 42
- B.1 The Jacobian, JVPs, and VJPs . . . 42
- B.2 Recovering multiple columns with a single evaluation . . . 43
- B.3 Sparsity patterns . . . 43
- B.4 Graph coloring . . . 43
- B.5 Decompression . . . 44
- B.6 The Hessian and HVPs . . . 44
- B.7 Star coloring: using symmetry to recover an entry from either side . . . 45
- B.8 From PET neighborhoods to a Hessian pattern . . . 45
- B.9 The complete SADMOF procedure in this project . . . 46
- B.10 Accuracy, truncation, and computational cost . . . 47

C Computational graphs, backpropagation, and parameter updates 48
- C.1 Step 1: construct and evaluate the computational graph . . . 48
- C.2 Step 2: propagate sensitivities in reverse order . . . 48
- C.3 A worked example: from forward values to parameter gradients . . . 49
- C.4 Step 3: collect gradients through network layers . . . 50
- C.5 Step 4: combine examples and update parameters . . . 50
- C.6 Shared weights, residual paths, and the PET computational graph . . . 50

C.7 From loss gradients to forces and second derivatives 51

Abstract

Methane-loaded MOF-5 heat capacity is estimated by combining classical NPT molecular dynamics with a loaded-system harmonic quantum correction from PET-JAX/SADMOF Hessians. PET-MAD and PET-SOL calculations at 50, 100, and 150 methane molecules give exploratory room-temperature values of about 0.93, 1.07–1.08, and 1.16 J/g/K. Sampling errors and paired last-layer prediction-rigidity (LLPR) model uncertainties are propagated separately. Production drift, imaginary modes, incomplete relaxation, and sensitivity to uncertainty propagation prevent a claim of convergence. Variation across quenched minima remains unestimated. The empty-framework Hessian provides a separate reference.

1 Scientific motivation and scope

Metal–organic frameworks (MOFs) are porous crystalline materials built from metal nodes and organic linkers. Their pore geometry and chemical composition can be varied to accommodate different guest molecules, making them attractive for gas storage, molecular separation, and water harvesting. These applications involve both the uptake of molecules and the exchange of heat: adsorption releases energy, while regeneration generally requires energy to remove the adsorbed guests. The heat capacity of the loaded material determines how much its temperature changes for a given heat input at fixed loading and contributes to the energy required to heat it during a working cycle. Calculating this property is therefore useful for connecting molecular structure to the thermal response of an adsorbent.

Guest adsorption also makes heat capacity a sensitive probe of microscopic motion. A loaded MOF combines framework vibrations, internal guest vibrations, and motion of molecules within the pores. Their contributions depend on temperature, loading, and interactions between the host and guests. High-frequency vibrations require quantum statistics, whereas soft framework motion and guest rearrangements can be strongly anharmonic. Capturing both effects at an affordable computational cost is the central challenge of this work [21].

This study evaluates a hybrid approach that combines classical molecular dynamics with a harmonic quantum correction, using the machine-learned potentials PET-MAD and PET-SOL for both contributions. Methane-loaded MOF-5 provides the initial test system and a connection to the study of Kapil *et al.* [21]. The objective is to assess the temperature and loading dependence of its heat capacity and establish the convergence and validity of the approach with PET. Extensions to carbon dioxide and water, and to MOF-303 and ZIF-8, will test how pore chemistry and framework flexibility affect the thermal response. In particular, MOF-303 offers polar water-binding sites [8], while ZIF-8 provides flexible pore windows [9]. The results presented here concern the MOF-5/methane comparison; the additional systems define the broader scope of the project.

2 Theory

2.1 Quantum Mechanics

2.1.1 States, eigenstates, and observables

A quantum state is represented by a ket $|\Psi\rangle$ in a Hilbert space; its Hermitian conjugate is the bra $\langle\Psi|$. The inner product $\langle\Phi|\Psi\rangle$ is a complex probability amplitude, and a normalized state satisfies $\langle\Psi|\Psi\rangle = 1$. Observables are represented by Hermitian operators, denoted with a hat [1]. For an observable $\hat{A}$, an orthonormal eigenbasis satisfies

$$\hat{A}|a_n\rangle = a_n|a_n\rangle, \qquad \langle a_m|a_n\rangle = \delta_{mn}, \qquad \sum_n |a_n\rangle\langle a_n| = \hat{I}. \tag{1}$$

The eigenvalue a_n is a possible measurement result. Expanding $|\Psi\rangle = \sum_n c_n|a_n\rangle$ gives probabilities $|c_n|^2 = |\langle a_n|\Psi\rangle|^2$ and the expectation value

$$\langle A\rangle_\Psi = \langle\Psi|\hat{A}|\Psi\rangle = \sum_n |c_n|^2 a_n. \tag{2}$$

Here sums refer to a discrete spectrum; continuous labels require integrals. We denote the Hamiltonian operator by $\hat{E}$, consistently reserving H for enthalpy. For a time-independent Hamiltonian, its energy eigenstates and the evolution of a general state obey

$$\hat{E}|\Psi_n\rangle = E_n|\Psi_n\rangle, \qquad i\hbar\frac{\partial}{\partial t}|\Psi(t)\rangle = \hat{E}|\Psi(t)\rangle. \tag{3}$$

These equations do not depend on a choice of basis.

2.1.2 The position representation

For the electron and nuclear positions collected in $\boldsymbol{r}$ and $\boldsymbol{R}$, the position eigenkets $|\boldsymbol{r},\boldsymbol{R}\rangle$ form a continuous basis. Spin labels are suppressed. The wavefunction is the projection of the abstract state onto this basis:

$$\Psi(\boldsymbol{r},\boldsymbol{R},t) = \langle\boldsymbol{r},\boldsymbol{R}|\Psi(t)\rangle, \qquad |\Psi(t)\rangle = \int \mathrm{d}\boldsymbol{r}\,\mathrm{d}\boldsymbol{R}\,|\boldsymbol{r},\boldsymbol{R}\rangle\Psi(\boldsymbol{r},\boldsymbol{R},t). \tag{4}$$

The integration covers all particle coordinates, and the bra has wavefunction $\langle\Psi|\boldsymbol{r},\boldsymbol{R}\rangle = \Psi^*(\boldsymbol{r},\boldsymbol{R})$. Position acts by multiplication and momentum by differentiation; for nucleus i,

$$\langle\boldsymbol{r},\boldsymbol{R}|\hat{\boldsymbol{R}}_i|\Psi\rangle = \boldsymbol{R}_i\Psi(\boldsymbol{r},\boldsymbol{R}), \qquad \langle\boldsymbol{r},\boldsymbol{R}|\hat{\boldsymbol{p}}_i|\Psi\rangle = -i\hbar\nabla_{\boldsymbol{R}_i}\Psi(\boldsymbol{r},\boldsymbol{R}). \tag{5}$$

For example, the pure-state expectation in Eq. (2) becomes

$$\langle A\rangle_\Psi = \int \mathrm{d}\boldsymbol{r}\,\mathrm{d}\boldsymbol{R}\,\Psi^*(\boldsymbol{r},\boldsymbol{R})\,A_{\mathrm{pos}}\Psi(\boldsymbol{r},\boldsymbol{R}), \tag{6}$$

where A_{pos} denotes the operator acting on wavefunctions in this basis. Neglecting relativistic effects, the electron–nuclear Hamiltonian is

$$\hat{E}_{\mathrm{tot}} = \hat{T}_{\mathrm{n}} + \hat{T}_{\mathrm{e}} + \hat{V}_{\mathrm{ee}} + \hat{V}_{\mathrm{en}} + \hat{V}_{\mathrm{nn}}. \tag{7}$$

Projecting its Schrödinger equation with $\langle\boldsymbol{r},\boldsymbol{R}|$ gives the position-space differential equation

$$i\hbar\frac{\partial\Psi}{\partial t} = \left[-\sum_i \frac{\hbar^2}{2m_i}\nabla^2_{\boldsymbol{R}_i} - \sum_a \frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2_{\boldsymbol{r}_a} + V_{\mathrm{ee}} + V_{\mathrm{en}} + V_{\mathrm{nn}}\right]\Psi. \tag{8}$$

The V terms here are coordinate-dependent Coulomb potentials. Thus a ket and its wavefunction describe the same state, while the differential operator is the position representation of the Hamiltonian. Solving the coupled many-electron and nuclear problem requires approximations; the first used here is the Born–Oppenheimer separation [2, 5].

2.1.3 The Born–Oppenheimer approximation

Because nuclei are much heavier than electrons, we approximate their motion on a single electronic ground-state potential-energy surface. At each fixed nuclear configuration $\boldsymbol{R}$, solve the electronic *operator* eigenvalue problem

$$\hat{E}_{\mathrm{el}}(\boldsymbol{R}) = \hat{T}_{\mathrm{e}} + \hat{V}_{\mathrm{ee}} + \hat{V}_{\mathrm{en}}(\boldsymbol{R}), \tag{9}$$

$$\hat{E}_{\mathrm{el}}(\boldsymbol{R})|\psi_0(\boldsymbol{R})\rangle = \varepsilon_0(\boldsymbol{R})|\psi_0(\boldsymbol{R})\rangle. \tag{10}$$

Here $\boldsymbol{R}$ is a parameter, while $|\psi_0(\boldsymbol{R})\rangle$ is a state in the electronic Hilbert space. Its position representation is $\psi_0(\boldsymbol{r};\boldsymbol{R}) = \langle \boldsymbol{r}|\psi_0(\boldsymbol{R})\rangle$. Projecting with $\langle \boldsymbol{r}|$ gives the equivalent equation

$$\left[-\sum_a \frac{\hbar^2}{2m_\mathrm{e}}\nabla^2_{\boldsymbol{r}_a} + V_\mathrm{ee}(\boldsymbol{r}) + V_\mathrm{en}(\boldsymbol{r};\boldsymbol{R})\right]\psi_0(\boldsymbol{r};\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R})\psi_0(\boldsymbol{r};\boldsymbol{R}). \tag{11}$$

The semicolon distinguishes the electronic coordinates from the nuclear parameters. Adding nuclear repulsion defines the potential-energy surface $U(\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R}) + V_\mathrm{nn}(\boldsymbol{R})$. Neglecting derivative couplings associated with the nuclear dependence of the electronic state gives the nuclear Hamiltonian

$$\hat{E}_\mathrm{nuc} = \sum_i \frac{|\hat{\boldsymbol{p}}_i|^2}{2m_i} + U(\hat{\boldsymbol{R}}). \tag{12}$$

Its eigenstates $|\chi_\nu\rangle$ and their nuclear wavefunctions $\chi_\nu(\boldsymbol{R}) = \langle \boldsymbol{R}|\chi_\nu\rangle$ satisfy, respectively,

$$\hat{E}_\mathrm{nuc}|\chi_\nu\rangle = E_\nu|\chi_\nu\rangle, \tag{13}$$

$$\left[-\sum_i \frac{\hbar^2}{2m_i}\nabla^2_{\boldsymbol{R}_i} + U(\boldsymbol{R})\right]\chi_\nu(\boldsymbol{R}) = E_\nu\chi_\nu(\boldsymbol{R}). \tag{14}$$

The corresponding approximation to the total wavefunction is a product *in the position representation*:

$$\Psi_\nu(\boldsymbol{r},\boldsymbol{R}) \approx \psi_0(\boldsymbol{r};\boldsymbol{R})\chi_\nu(\boldsymbol{R}). \tag{15}$$

The electronic factor still depends on the nuclear configuration. Classical MD later replaces this quantum nuclear evolution by motion of positions and momenta on the same surface U.

2.2 Density Functional Theory

At fixed nuclear positions, density functional theory (DFT) expresses the electronic ground-state energy in terms of the electron density $n(\boldsymbol{r})$ rather than the many-electron wavefunction. This provides the Born–Oppenheimer potential-energy surface used to generate reference energies, forces, and stresses for machine-learned potentials.

2.2.1 The Hohenberg–Kohn theorems

For a fixed electron number N_e and external potential $v_\mathrm{ext}(\boldsymbol{r};\boldsymbol{R})$, the electronic Hamiltonian is

$$\hat{E}_\mathrm{el} = \hat{T}_\mathrm{e} + \hat{V}_\mathrm{ee} + \sum_{a=1}^{N_\mathrm{e}} v_\mathrm{ext}(\hat{\boldsymbol{r}}_a;\boldsymbol{R}). \tag{16}$$

The Hohenberg–Kohn (HK) theorems establish the basis of ground-state DFT [6]:

1. The ground-state density $n_0(\boldsymbol{r})$ determines the external potential up to an additive constant, and hence the electronic Hamiltonian and ground-state properties.
2. The ground-state energy is the minimum of the density functional $E_v[n]$, attained at n_0, over admissible densities with $\int n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r} = N_\mathrm{e}$.

Writing the universal functional $F[n]$ in constrained-search form makes this variational principle explicit [5]:

$$F[n] = \min_{\psi\to n}\langle\psi|\hat{T}_\mathrm{e} + \hat{V}_\mathrm{ee}|\psi\rangle, \tag{17}$$

$$E_v[n] = F[n] + \int v_\mathrm{ext}(\boldsymbol{r};\boldsymbol{R})n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r}, \qquad E_0 = \min_{n\ \text{admissible}} E_v[n]. \tag{18}$$

HK therefore turns the electronic problem into a density minimization, but does not supply a practical expression for $F[n]$.

2.2.2 From HK to the Kohn–Sham equations

Kohn–Sham (KS) theory introduces an auxiliary system of noninteracting electrons with the same density [7]. Its orthonormal orbitals $|\phi_a\rangle$, written as $\phi_a(\boldsymbol{r}) = \langle \boldsymbol{r}|\phi_a\rangle$ in the one-electron position basis, and occupations f_a give

$$n(\boldsymbol{r}) = \sum_a f_a|\phi_a(\boldsymbol{r})|^2, \qquad \sum_a f_a = N_\mathrm{e}. \tag{19}$$

The occupations include spin multiplicity; for a periodic solid the sum also includes band indices and wavevector weights. The unknown HK functional is decomposed as

$$F[n] = T_{\mathrm{s}}[n] + E_{\mathrm{H}}[n] + E_{\mathrm{xc}}[n], \tag{20}$$

$$T_{\mathrm{s}}[n] = \sum_a f_a \int \phi_a^*(\boldsymbol{r}) \left(-\frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2\right) \phi_a(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r}, \tag{21}$$

$$E_{\mathrm{H}}[n] = \frac{1}{2}\iint n(\boldsymbol{r}) w(\boldsymbol{r},\boldsymbol{r}') n(\boldsymbol{r}')\,\mathrm{d}\boldsymbol{r}\,\mathrm{d}\boldsymbol{r}'. \tag{22}$$

Here T_{s} is the noninteracting kinetic energy and E_{H} is the classical electron–electron electrostatic energy. For the periodic MOF, w is the periodic Coulomb kernel. The exchange–correlation functional E_{xc} contains the remaining interaction energy and the correction to T_{s}. Thus the electronic energy functional becomes

$$E_{\mathrm{el}}[n;\boldsymbol{R}] = T_{\mathrm{s}}[n] + \int v_{\mathrm{ext}}(\boldsymbol{r};\boldsymbol{R}) n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r} + E_{\mathrm{H}}[n] + E_{\mathrm{xc}}[n]. \tag{23}$$

Minimizing this energy with respect to the orbitals while preserving their orthonormality gives the one-electron KS equations:

$$\left[-\frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2 + v_{\mathrm{eff}}[n](\boldsymbol{r})\right] \phi_a(\boldsymbol{r}) = \epsilon_a \phi_a(\boldsymbol{r}), \tag{24}$$

$$v_{\mathrm{eff}}[n](\boldsymbol{r}) = v_{\mathrm{ext}}(\boldsymbol{r}) + v_{\mathrm{H}}[n](\boldsymbol{r}) + v_{\mathrm{xc}}[n](\boldsymbol{r}), \tag{25}$$

$$v_{\mathrm{H}}[n](\boldsymbol{r}) = \int w(\boldsymbol{r},\boldsymbol{r}') n(\boldsymbol{r}')\,\mathrm{d}\boldsymbol{r}', \qquad v_{\mathrm{xc}}[n](\boldsymbol{r}) = \frac{\delta E_{\mathrm{xc}}}{\delta n(\boldsymbol{r})}. \tag{26}$$

The decomposition is formally exact when the auxiliary system can reproduce the density; practical DFT approximates E_{xc}. Because v_{eff} depends on n, the equations are solved self-consistently: start from a trial density, solve for the orbitals, rebuild the density, and iterate until both density and energy converge.

At the converged density n_0, adding nuclear repulsion gives the potential energy and its derivatives give the reference forces:

$$U_{\mathrm{DFT}}(\boldsymbol{R}) = E_{\mathrm{el}}[n_0;\boldsymbol{R}] + V_{\mathrm{nn}}(\boldsymbol{R}), \qquad \boldsymbol{F}_i^{\mathrm{DFT}} = -\nabla_{\boldsymbol{R}_i} U_{\mathrm{DFT}}|_{\boldsymbol{h}}\,. \tag{27}$$

Cell-strain derivatives similarly provide stresses. These quantities supply training labels for MLIPs such as PET-MAD [24]. In this project, the MLIP then evaluates the potential-energy surface during MD and Hessian calculations without repeating the electronic calculation.

2.3 Statistical Mechanics

The classical simulations in this project use the isothermal–isobaric (NPT) ensemble. The numbers of framework atoms and guest molecules are fixed, while the system exchanges energy and volume with reservoirs at temperature T and imposed hydrostatic pressure P. Allowing the periodic cell to fluctuate captures the volume response of the loaded framework. The quantity of interest is therefore the constant-pressure heat capacity at fixed loading [3, 4].

For N classical atoms, write the positions and momenta explicitly as $\boldsymbol{r}^N = (\boldsymbol{r}_1,\ldots,\boldsymbol{r}_N)$ and $\boldsymbol{p}^N = (\boldsymbol{p}_1,\ldots,\boldsymbol{p}_N)$. The cell dependence of functions is left implicit throughout this section: they are evaluated in the instantaneous periodic cell, with $V = \det \boldsymbol{h}$. The cell remains an independent fluctuating variable; positions and momenta alone do not determine its volume. We denote the physical Hamiltonian by E, reserving H for enthalpy:

$$E(\boldsymbol{r}^N,\boldsymbol{p}^N) = K(\boldsymbol{p}^N) + U(\boldsymbol{r}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{r}^N), \tag{28}$$

where U is the MLIP potential energy. The instantaneous physical enthalpy is

$$H(\boldsymbol{r}^N,\boldsymbol{p}^N) = E(\boldsymbol{r}^N,\boldsymbol{p}^N) + PV = \sum_{i=1}^{N} \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{r}^N) + PV. \tag{29}$$

It uses the imposed pressure and excludes thermostat and barostat energies.

With $\beta = (k_{\mathrm{B}}T)^{-1}$, the NPT partition function and probability density are

$$\Delta(N,P,T) = \mathcal{C}\int_0^\infty \frac{\mathrm{d}V}{V_0} \int_{\mathrm{cell}(V)} \mathrm{d}^{3N}r \int \mathrm{d}^{3N}p\, e^{-\beta H(\boldsymbol{r}^N,\boldsymbol{p}^N)}, \tag{30}$$

$$\rho(\boldsymbol{r}^N,\boldsymbol{p}^N) = \Delta^{-1} e^{-\beta H(\boldsymbol{r}^N,\boldsymbol{p}^N)}. \tag{31}$$

Here V_0 is a fixed, temperature- and pressure-independent reference volume, and $\mathcal{C}$ contains the fixed phase-space normalization factors needed to make Δ dimensionless. The position integral covers the instantaneous cell for every atom, and the momentum integral covers the allowed momenta. The volume integral is retained because the cell fluctuates in NPT. Although the cell argument is suppressed in ρ, this is the joint density of atomic positions, momenta, and cell volume, rather than a density with the volume already integrated out. Every equilibrium observable is averaged as

$$\langle A \rangle = \mathcal{C} \int_0^\infty \frac{\mathrm{d}V}{V_0} \int_{\mathrm{cell}(V)} \mathrm{d}^{3N} r \int \mathrm{d}^{3N} p \, \rho(\boldsymbol{r}^N, \boldsymbol{p}^N) A(\boldsymbol{r}^N, \boldsymbol{p}^N), \tag{32}$$

Throughout this report, P denotes the imposed hydrostatic pressure, and equilibrium averages, variances, and covariances refer to the NPT ensemble unless stated otherwise. Ensemble subscripts are omitted.

The corresponding thermodynamic potential is the Gibbs free energy. For a temperature-independent Hamiltonian,

$$G = -k_\mathrm{B} T \ln \Delta, \qquad \langle H \rangle = -\left(\frac{\partial \ln \Delta}{\partial \beta} \right)_{N,P}, \tag{33}$$

$$\langle V \rangle = \left(\frac{\partial G}{\partial P} \right)_{N,T}, \qquad S = -\left(\frac{\partial G}{\partial T} \right)_{N,P}, \tag{34}$$

$$\langle E \rangle = \langle H \rangle - P \langle V \rangle, \qquad G = \langle H \rangle - TS. \tag{35}$$

The volume integral above displays the scalar-volume formulation. In the actual simulations the cell matrix $\boldsymbol{h}$ also permits anisotropic strain under hydrostatic external pressure, with $V = \det \boldsymbol{h}$. Fractional coordinates $\boldsymbol{s}_i = \boldsymbol{h}^{-1} \boldsymbol{r}_i$ give $\mathrm{d}^{3N} r = V^N \mathrm{d}^{3N} s$; integrating out the momenta therefore gives a configurational weight proportional to $V^N e^{-\beta[U+PV]}$ in fractional coordinates. Flexible-cell sampling must include the appropriate cell measure and extended-system corrections [27].

Additionally, we assume the ergodic hypothesis, which states that the infinite time average of an observable along a trajectory is equal to its ensemble average. So the integral in Eq. (32) can be replaced by a time average along a sufficiently long trajectory:

$$\langle A \rangle = \lim_{\tau \to \infty} \frac{1}{\tau} \int_0^\tau A\big(\boldsymbol{r}^N(t), \boldsymbol{p}^N(t)\big) \, \mathrm{d}t. \tag{36}$$

In practice, we can't run the simulation for ever, so we need to run the simulation for long enough to ensure that the system has reached equilibrium and explored the relevant phase space. We can then replace the integral by a finite sum over the trajectory frames, giving the time average

$$\langle A \rangle \approx \frac{1}{M} \sum_{j=1}^{M} A\big(\boldsymbol{r}^N(t_j), \boldsymbol{p}^N(t_j)\big), \tag{37}$$

2.3.1 Equipartition and temperature

The classical equipartition theorem relates temperature to the mean energy stored in quadratic degrees of freedom: each independent term of the form $ax^2/2$, with $a > 0$, contributes $k_\mathrm{B}T/2$. The coefficient changes the amplitude of the fluctuations, but not their mean energy. This provides both a temperature estimator for MD and the classical reference for vibrational heat capacity [3, 4].

To derive the result, take an unconstrained momentum component or a harmonic displacement x ranging over the real line. Its conditional equilibrium density is proportional to $e^{-\beta a x^2/2}$: the remaining energy terms, including PV, are independent of x and cancel in the normalized average. Integration by parts gives

$$0 = \int_{-\infty}^{\infty} \frac{\mathrm{d}}{\mathrm{d}x} \left(x e^{-\beta a x^2/2} \right) \mathrm{d}x = \int_{-\infty}^{\infty} (1 - \beta a x^2) e^{-\beta a x^2/2} \, \mathrm{d}x, \tag{38}$$

$$\left\langle \frac{a x^2}{2} \right\rangle = \frac{1}{2\beta} = \frac{1}{2} k_\mathrm{B} T. \tag{39}$$

The boundary term vanishes because the Gaussian decays at infinity. The same result holds after averaging over the other variables, including the fluctuating cell. Its application to potential energy requires the quadratic approximation; its application to the separable kinetic energy does not require harmonic atomic interactions.

For an atomic momentum component, $x = p_{i\alpha}$ and $a = 1/m_i$. Summing over f active momentum degrees of freedom gives

$$\langle K \rangle = \frac{f}{2} k_\mathrm{B} T, \qquad T_\mathrm{kin} = \frac{2K}{f k_\mathrm{B}}. \tag{40}$$

Subtract constraints and deliberately removed momentum components from $3N$; fixing total linear momentum gives $f = 3N - 3$. Auxiliary reservoir variables do not count. Initial velocities follow the Maxwell–Boltzmann distribution:

$$v_{i\alpha} \sim \mathcal{N}\left(0, \frac{k_\mathrm{B}T}{m_i}\right), \qquad \alpha \in \{x, y, z\}. \tag{41}$$

Subtract center-of-mass velocity when required. The NPT momentum distribution gives $\mathrm{var}(K) = f(k_\mathrm{B}T)^2/2$ and $\mathrm{var}(T_\mathrm{kin}) = 2T^2/f$; repeated exact rescaling suppresses these fluctuations.

In this project, these relations set the initial velocity distribution and the kinetic-energy target of the thermostat, and provide a check on sampled temperature fluctuations. For each harmonic mode, equipartition assigns $k_\mathrm{B}T/2$ to kinetic energy and the same amount to potential energy, giving a classical heat capacity of k_B per mode. This is the harmonic contribution subtracted in the hybrid correction. Quantum modes with $\hbar\omega \geq k_\mathrm{B}T$, especially bond stretches, contribute less than this classical value. Section 2.3.3 derives their quantum contribution and the Dulong–Petit limit. The full classical C_P still comes from the sampled enthalpy: equipartition alone does not determine anharmonic or pressure–volume contributions.

2.3.2 Heat capacity from mean enthalpy

At fixed composition and imposed pressure, heat capacity measures the change in equilibrium mean enthalpy with temperature:

$$C_P(T) = \left(\frac{\partial\langle H\rangle}{\partial T}\right)_{N,P}, \qquad \langle H\rangle_T = \langle K + U + PV\rangle_T. \tag{42}$$

The project estimates this derivative from separate equilibrated MD trajectories at neighboring temperatures. On a uniformly spaced grid, the centered finite difference is

$$C_P(T) \simeq \frac{\langle H\rangle_{T+\Delta T} - \langle H\rangle_{T-\Delta T}}{2\Delta T}. \tag{43}$$

All trajectories entering the difference use the same composition, model, and imposed pressure. Each mean includes kinetic, potential, and pressure–volume contributions, retaining the anharmonic behavior sampled by classical MD. The mass-specific heat capacity is

$$c_P(T) = \frac{C_P(T)}{M_\mathrm{cell}}, \tag{44}$$

where M_cell includes both framework and guest mass.

For a smooth enthalpy curve, the centered approximation has a truncation error of order $(\Delta T)^2$. Smaller spacing reduces this error but amplifies sampling noise in the difference, so both trajectory sampling and temperature spacing require convergence checks. The production grid uses $\Delta T = 25\,\mathrm{K}$ and includes neighboring temperatures beyond the reported interval to retain centered derivatives at its endpoints (Section 3.4).

2.3.3 Quantum harmonic heat capacity and the Dulong–Petit law

Quantum statistical mechanics averages over populations of states as well as the measurement probabilities within each state. Here $\hat{E}$ is the Born–Oppenheimer nuclear Hamiltonian of Eq. (12); its position variables are the atomic coordinates denoted $\boldsymbol{r}^N$ in the classical discussion. At a fixed cell, let $\hat{E}|\chi_\nu\rangle = E_\nu|\chi_\nu\rangle$. If the system occupies energy eigenstate $|\chi_\nu\rangle$ with probability w_ν, then

$$\langle A\rangle_\mathrm{q} = \sum_\nu w_\nu \langle\chi_\nu|\hat{A}|\chi_\nu\rangle, \qquad \hat{\rho}_\mathrm{q} = \sum_\nu w_\nu |\chi_\nu\rangle\langle\chi_\nu|, \qquad \sum_\nu w_\nu = 1. \tag{45}$$

The density operator $\hat{\rho}_\mathrm{q}$ packages the statistical mixture [1]. For any complete orthonormal basis $\{|m\rangle\}$, define $\mathrm{Tr}\,\hat{B} = \sum_m \langle m|\hat{B}|m\rangle$. Inserting the density operator and using completeness gives

$$\mathrm{Tr}(\hat{\rho}_\mathrm{q}\hat{A}) = \sum_{m,\nu} w_\nu \langle m|\chi_\nu\rangle\langle\chi_\nu|\hat{A}|m\rangle \tag{46}$$

$$= \sum_\nu w_\nu \langle\chi_\nu|\hat{A}|\chi_\nu\rangle = \langle A\rangle_\mathrm{q}. \tag{47}$$

The trace is independent of basis, and $\hat{A}$ need not commute with $\hat{E}$. In the nuclear position basis the same average is

$$\langle A\rangle_\mathrm{q} = \int \mathrm{d}^{3N}r\, \langle\boldsymbol{r}^N|\hat{\rho}_\mathrm{q}\hat{A}|\boldsymbol{r}^N\rangle. \tag{48}$$

For a position-dependent observable, this reduces to the integral of $A(\boldsymbol{r}^N)$ weighted by $\langle\boldsymbol{r}^N|\hat{\rho}_\mathrm{q}|\boldsymbol{r}^N\rangle$.

At fixed cell and temperature, the Boltzmann populations give

$$w_\nu = \frac{e^{-\beta E_\nu}}{Z_\mathrm{q}}, \qquad Z_\mathrm{q} = \sum_\nu e^{-\beta E_\nu} = \mathrm{Tr}\, e^{-\beta \hat{E}}, \qquad \hat{\rho}_\mathrm{q} = \frac{e^{-\beta \hat{E}}}{Z_\mathrm{q}}. \tag{49}$$

The operator exponential is diagonal in the energy eigenbasis, which explains the equivalence between the sum and trace forms of the partition function. For a fluctuating cell, include the pressure work and integrate over volume as in the classical ensemble:

$$\Delta_\mathrm{q} = \int_0^\infty \frac{\mathrm{d}V}{V_0}\ \mathrm{Tr}\, e^{-\beta(\hat{E}+PV)}, \qquad \hat{\rho} = \Delta_\mathrm{q}^{-1} e^{-\beta(\hat{E}+PV)}, \tag{50}$$

$$\langle A \rangle = \int_0^\infty \frac{\mathrm{d}V}{V_0}\ \mathrm{Tr}(\hat{\rho}\hat{A}), \qquad 1 = \int_0^\infty \frac{\mathrm{d}V}{V_0}\,\mathrm{Tr}\,\hat{\rho}. \tag{51}$$

Here PV multiplies the identity operator and the cell dependence remains implicit. The trace is over nuclear states at each cell; the volume integral also averages over cells. For this project, the quantum correction is evaluated at a fixed optimized cell with fixed harmonic frequencies. The following mode averages therefore give C_V, while the classical MD contribution remains at constant pressure.

Expanding the potential about a stable minimum and using mass-weighted normal coordinates Q_k gives independent harmonic oscillators:

$$\hat{E}_\mathrm{har} = U_\mathrm{min} + \sum_{k\in\mathcal{V}} \left(\frac{\hat{\Pi}_k^2}{2} + \frac{\omega_k^2 \hat{Q}_k^2}{2} \right), \qquad \epsilon_{k,n} = \hbar\omega_k \left(n + \frac{1}{2} \right), \quad n = 0, 1, \ldots. \tag{52}$$

Here $\mathcal{V}$ contains the retained positive-frequency modes and Π_k is conjugate to Q_k. Each mode has partition function

$$z_k = \sum_{n=0}^\infty e^{-\beta\epsilon_{k,n}} = \frac{e^{-x_k/2}}{1-e^{-x_k}}, \qquad x_k = \beta\hbar\omega_k, \qquad Z_\mathrm{har} = e^{-\beta U_\mathrm{min}} \prod_{k\in\mathcal{V}} z_k. \tag{53}$$

Differentiating its logarithm gives the mean energy:

$$\langle E \rangle_\mathrm{har} = -\frac{\partial \ln Z_\mathrm{har}}{\partial\beta} = U_\mathrm{min} + \sum_{k\in\mathcal{V}} \hbar\omega_k \left(\frac{1}{2} + \frac{1}{e^{x_k}-1} \right). \tag{54}$$

The zero-point energy $\hbar\omega_k/2$ is independent of temperature at fixed frequencies and does not contribute to the heat capacity. Differentiating the thermal occupation gives

$$C_{V,\mathrm{q}}^\mathrm{har} = \left(\frac{\partial \langle E \rangle_\mathrm{har}}{\partial T} \right)_V = \sum_{k\in\mathcal{V}} k_\mathrm{B} \frac{x_k^2 e^{x_k}}{(e^{x_k}-1)^2}. \tag{55}$$

Equivalently, the contribution of one mode and the mass-specific sum are

$$C_{V,k}(T) = k_\mathrm{B} \left[\frac{x_k/2}{\sinh(x_k/2)} \right]^2, \qquad x_k = \frac{\hbar\omega_k}{k_\mathrm{B}T} = \frac{h_\mathrm{P} c \widetilde{\nu}_k}{k_\mathrm{B}T}, \tag{56}$$

$$c_V^\mathrm{har}(T) = \frac{1}{M_\mathrm{cell}} \sum_{k\in\mathcal{V}} C_{V,k}(T). \tag{57}$$

Here $\widetilde{\nu}_k$ is the mode wavenumber, h_P is Planck's constant, and c is the speed of light. Dividing a cell heat capacity in J/K by its mass in grams gives J/g/K; equivalently, one can multiply by $N_\mathrm{A}/M_\mathrm{molar}$. For a loaded system, the mass includes both the framework and all guest molecules. Modes with $\hbar\omega_k \gg k_\mathrm{B}T$ are thermally suppressed, whereas low-frequency modes approach their classical contribution.

In the high-temperature limit, $x_k \ll 1$, using $e^{x_k} - 1 \simeq x_k$ gives $C_{V,k} \to k_\mathrm{B}$. Classical equipartition gives the same result: each oscillator has one quadratic kinetic and one quadratic potential term, together contributing $k_\mathrm{B}T$ to its mean energy. Thus

$$C_{V,\mathrm{cl}}^\mathrm{har} = |\mathcal{V}| k_\mathrm{B}, \qquad C_V \longrightarrow 3Nk_\mathrm{B}, \qquad C_{V,\mathrm{molar}} \longrightarrow 3R, \quad R = N_\mathrm{A} k_\mathrm{B}. \tag{58}$$

The last two expressions are the Dulong–Petit law for a bulk solid with N atoms, with molar heat capacity defined per mole of atoms. A finite periodic-cell calculation omitting the three uniform translations instead has $3N-3$ vibrational modes before any additional mode filtering. For the loaded MOF, N includes both framework and guest atoms; the mass-specific limit is $|\mathcal{V}|k_\mathrm{B}/M_\mathrm{cell}$.

2.3.4 Hybrid quantum–classical approximation

The hybrid approximation of Kapil *et al.* [21] replaces the classical harmonic contribution by its quantum counterpart while retaining the anharmonic contribution sampled by classical MD:

$$C_P^{\mathrm{hyb}}(T) \simeq C_{P,\mathrm{cl}}^{\mathrm{anh}}(T) + C_{V,\mathrm{qn}}^{\mathrm{har}}(T) - C_{V,\mathrm{cl}}^{\mathrm{har}}(T). \tag{59}$$

Here $C_{P,\mathrm{cl}}^{\mathrm{anh}}$ denotes the full classical heat capacity obtained from MD on the anharmonic potential. The harmonic quantum and classical terms use the same retained mode set $\mathcal{V}$, so their difference is

$$\Delta C^{\mathrm{har}}(T) = \sum_{k\in\mathcal{V}} k_{\mathrm{B}} \left[\frac{x_k^2 e^{x_k}}{(e^{x_k}-1)^2} - 1 \right]. \tag{60}$$

This correction tends to zero in the classical high-temperature limit. For an adsorbate-loaded framework, it must be evaluated from the complete loaded-system Hessian: an empty-framework spectrum omits guest vibrations and their coupling to the host.

The approximation treats the quantum correction harmonically and the remaining anharmonicity classically. Adding a fixed-cell C_V correction to the MD C_P further assumes that the quantum change in $C_P - C_V$ is small. Reusing one reference spectrum over a temperature interval also neglects changes in its force constants and the distribution of occupied minima. These assumptions require validation for each host, guest, loading, and potential.

Kapil *et al.* used a DFT-derived QuickFF force field [26] and found the hybrid estimate useful above approximately 200 K in their 100-methane MOF-5 test. It qualitatively reproduced a loading-dependent heat-capacity minimum associated with changes in host–guest interactions. This result motivates the temperature range of the present study, but does not establish the same validity range for PET.

2.3.5 Classical MD and quantum nuclear statistics

On the Born–Oppenheimer surface introduced in Section 2.1.3, classical MD replaces nuclear operators by positions and momenta, giving the Hamiltonian in Eq. (28) and Newtonian forces $m_i\ddot{\boldsymbol{R}}_i = -\nabla_i U$. A useful harmonic criterion and thermal length scale are

$$\beta\hbar\omega \ll 1, \qquad \lambda_i = \sqrt{\frac{2\pi\hbar^2}{m_i k_{\mathrm{B}} T}}, \qquad \beta = (k_{\mathrm{B}}T)^{-1}. \tag{61}$$

Classical sampling is appropriate when quantum level spacing is small relative to thermal energy and nuclear delocalization is negligible on relevant structural scales. A hydrogen-containing system can satisfy these conditions for slow collective modes while violating them for bond vibrations.

The fixed-cell quantum averages are given by Eqs. (47) and (49). The harmonic derivation in Section 2.3.3 provides the quantum nuclear populations used in this project, while MD samples the classical anharmonic surface. Quantum anharmonic sampling is an extension beyond the reported calculations.

2.3.6 Trajectory analysis and error estimation

Only equilibrated trajectory segments are used for averages. Energy, volume, temperature, and structural observables should fluctuate without systematic drift before data collection begins. For M equally spaced samples of an observable A, the time average is

$$\overline{A} = \frac{1}{M}\sum_{n=1}^{M} A_n. \tag{62}$$

Successive samples are correlated. Their autocovariance and normalized autocorrelation at lag k are

$$C_A(k) = \langle (A_n - \langle A\rangle)(A_{n+k} - \langle A\rangle)\rangle, \tag{63}$$

$$\rho_A(k) = C_A(k)/C_A(0). \tag{64}$$

These correlations determine how much independent information a trajectory contains. The sample-mean variance is

$$\mathrm{var}(\overline{A}) = \frac{C_A(0)}{M}\left[1 + 2\sum_{k=1}^{M-1}\left(1 - \frac{k}{M}\right)\rho_A(k)\right]. \tag{65}$$

Estimate or truncate the correlation sum statistically. Effective sample counts depend on the observable; finer output stride does not add independent information over a fixed duration.

Block errors should plateau for blocks longer than the correlation time, while retaining enough blocks [15]. Independent replicas help detect slow modes, but neither method reveals a basin never visited. Resample whole replicas or sufficiently long blocks and recompute the full estimator, preserving cross-temperature correlations. Variance-based heat capacities require errors of squared fluctuations, not merely errors of the mean enthalpy. Sampling error, finite-size effects, integration bias, and potential error need separate checks.

2.4 Molecular dynamics

MD turns the potential-energy surface into a trajectory of atomic positions and momenta. The framework and guests evolve together, sampling vibrations, guest rearrangements, and cell deformation. Evaluating the full potential at every step includes classical anharmonic motion. Its equilibrium averages provide the enthalpies needed for C_P; the separate harmonic calculation supplies the quantum correction [3, 4].

2.4.1 Forces and time integration

At fixed cell, Newton's equations are

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{p}_i}{m_i}, \qquad \dot{\boldsymbol{p}}_i = \boldsymbol{F}_i, \qquad \boldsymbol{F}_i = -\nabla_{\boldsymbol{R}_i} U(\boldsymbol{R}; \boldsymbol{h}). \tag{66}$$

Here $\boldsymbol{R}$ collects all atomic positions, using the same coordinates written as $\boldsymbol{r}^N$ in Section 2.3. A force evaluation supplies the accelerations, which advance the system to its next configuration. Velocity Verlet implements this cycle with two momentum half steps:

$$\boldsymbol{p}_i^{n+1/2} = \boldsymbol{p}_i^n + \frac{\Delta t}{2} \boldsymbol{F}_i(\boldsymbol{R}^n), \tag{67}$$

$$\boldsymbol{R}_i^{n+1} = \boldsymbol{R}_i^n + \Delta t \frac{\boldsymbol{p}_i^{n+1/2}}{m_i}, \tag{68}$$

$$\boldsymbol{p}_i^{n+1} = \boldsymbol{p}_i^{n+1/2} + \frac{\Delta t}{2} \boldsymbol{F}_i(\boldsymbol{R}^{n+1}). \tag{69}$$

The method is second order, reversible, and symplectic for the fixed-cell Hamiltonian problem, requiring one new force evaluation per step. The production integrator adds thermostat and cell updates to this splitting. Its timestep must resolve the fastest flexible vibrations, particularly X–H stretches. For a harmonic mode, $\omega_{\max} \Delta t < 2$ is a stability condition, not an accuracy test: enthalpies and structural observables must also converge with timestep. Short fixed-cell energy-conservation tests help detect integration errors that a thermostat could otherwise mask.

2.4.2 Periodic geometry and interaction range

A periodic cell represents a repeating bulk material (Figure 1). With cell matrix $\boldsymbol{h}$, $V = \det \boldsymbol{h}$ and $\boldsymbol{R}_i = \boldsymbol{h}\boldsymbol{s}_i$, where $\boldsymbol{s}_i$ is fractional position. Periodic neighbors are separated by

$$\boldsymbol{r}_{ij\boldsymbol{n}} = \boldsymbol{R}_j - \boldsymbol{R}_i + \boldsymbol{h}\boldsymbol{n}, \qquad \boldsymbol{n} \in \mathbb{Z}^3. \tag{70}$$

Thus atoms near opposite faces can be close neighbors. In a one-dimensional cell of length 20 Å, positions 1 and 19 Å are only 2 Å apart through a periodic image. Crossing a boundary changes an atom's displayed image; unwrapped positions retain its displacement history.

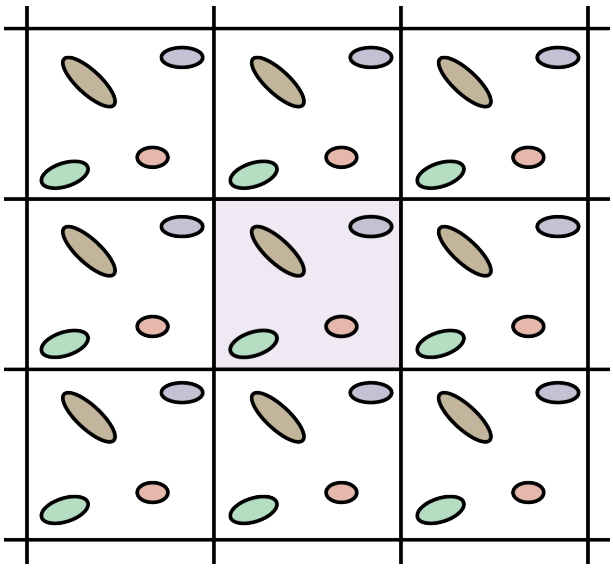


Figure 1: The central simulation cell and its periodic images in two dimensions. Neighbors can lie across a cell boundary. Reproduced unchanged from Christopher Rowley [10], CC BY-SA 4.0.

Periodicity specifies geometry, not interaction range. The present PET model uses local, many-body neighborhoods; there is no separate long-range electrostatic solver in the MD input. For a short enough cutoff, the closest

periodic image suffices. Otherwise multiple images within the cutoff must be included. A neighbor list adds a skin beyond the physical cutoff and is rebuilt as atoms or the cell move. Smooth cutoffs and correct triclinic image handling are essential for continuous forces.

2.4.3 Temperature and pressure control

Newtonian dynamics in a fixed periodic cell conserves the physical Hamiltonian $E = K+U$: it describes an isolated system at fixed volume. Assigning initial velocities sets an initial kinetic temperature, but does not maintain the desired temperature as potential and kinetic energy interconvert. For this project, we instead need equilibrium averages at prescribed T and P, including thermal expansion of the porous framework. A thermostat represents heat exchange with a reservoir; a barostat allows the cell to exchange mechanical work with its surroundings. Together they sample the distribution proportional to $\exp[-\beta(E+PV)]$ introduced in Section 2.3.

A thermostat couples the atomic momenta to auxiliary dynamical variables. A single Nosé–Hoover thermostat introduces an auxiliary friction variable:

$$\dot{\boldsymbol{p}}_i = \boldsymbol{F}_i - \xi \boldsymbol{p}_i, \qquad \dot{\xi} = \frac{2K - f k_\mathrm{B} T}{Q}, \qquad Q \sim f k_\mathrm{B} T \tau_T^2. \tag{71}$$

Here f is the number of active kinetic degrees of freedom, Q is the thermostat inertia, and τ_T is its characteristic response time. The sign of the feedback removes excess kinetic energy or restores it. For example, $2K > f k_\mathrm{B} T$ increases ξ, producing friction. Nosé–Hoover chains also thermostat the auxiliary variables, improving sampling of nearly harmonic systems [28]. The target temperature sets an equilibrium distribution, so instantaneous temperature should fluctuate rather than remain exactly fixed.

The physical potential U is unchanged, but the atomic system now exchanges energy with additional degrees of freedom. Introduce a dimensionless thermostat coordinate η and its momentum $p_\eta = Q\dot{\eta} = Q\xi$. The corresponding extended energy is

$$E_{\mathrm{ext},T}(\boldsymbol{R}, \boldsymbol{p}, \eta, p_\eta) = \underbrace{\sum_i \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{R})}_{E} + \underbrace{\frac{p_\eta^2}{2Q}}_{\text{reservoir kinetic energy}} + \underbrace{f k_\mathrm{B} T \eta}_{\text{reservoir potential}}. \tag{72}$$

Energy lost by the atoms is gained by the reservoir, giving $\mathrm{d}E_{\mathrm{ext},T}/\mathrm{d}t = 0$ for the continuous equations. This quantity is often called an extended Hamiltonian. In the physical-time Nosé–Hoover formulation used here, it is a conserved energy rather than a canonical Hamiltonian generating all the equations through ordinary Hamilton equations [28, 12].

The barostat responds to instantaneous pressure, which contains kinetic and configurational contributions:

$$\boldsymbol{P}_\mathrm{inst} = \frac{1}{V}\left[\sum_i \frac{\boldsymbol{p}_i \otimes \boldsymbol{p}_i}{m_i} + \boldsymbol{W}\right], \qquad P_\mathrm{inst} = \frac{1}{3}\operatorname{tr} \boldsymbol{P}_\mathrm{inst}. \tag{73}$$

For the many-body model, the virial follows from the energy response to strain at fixed fractional positions:

$$W_{\alpha\beta} = -\left.\frac{\partial U}{\partial \epsilon_{\alpha\beta}}\right|_{\boldsymbol{\epsilon}=0, \{\boldsymbol{s}_i\}}, \qquad \boldsymbol{h} \mapsto (\boldsymbol{I} + \boldsymbol{\epsilon})\boldsymbol{h}. \tag{74}$$

The model's configurational stress excludes the kinetic term; tensile-positive stress has the opposite sign to pressure [11]. Hydrostatic loading imposes $P\boldsymbol{I}$ with zero shear target. A flexible barostat permits independent cell lengths and tilts, so an isotropic pressure need not produce isotropic deformation. Cell changes carry atoms through $\boldsymbol{R}_i = \boldsymbol{h}\boldsymbol{s}_i$, alongside their motion within the cell. The MTK corrections ensure the intended ensemble measure [27, 12].

The barostat adds the cell matrix $\boldsymbol{h}$ and six generalized cell momenta $\pi_a = W_a \nu_a$, where ν_a are the three normal and three shear strain-rate variables and W_a their fictitious inertias. Pressure imbalance drives these variables: excess internal pressure drives expansion, while pressure below the target drives contraction. Thermostat chains introduce coordinates and momenta (η_j, p_{η_j}) for the atoms and (ζ_j, p_{ζ_j}) for the barostat, with inertias Q_j and Q'_j. For the project's two chains of length three and six independently barostatted cell components, the extended energy becomes

$$\begin{aligned} E_{\mathrm{ext},TP} = {} & \underbrace{\sum_i \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{R}; \boldsymbol{h})}_{E} + \underbrace{P \det \boldsymbol{h}}_{PV} + \underbrace{\sum_{a=1}^{6} \frac{\pi_a^2}{2W_a}}_{\text{cell kinetic energy}} \\ & + \sum_{j=1}^{3}\left(\frac{p_{\eta_j}^2}{2Q_j} + \frac{p_{\zeta_j}^2}{2Q'_j}\right) \\ & + k_\mathrm{B} T\left(f\eta_1 + \eta_2 + \eta_3 + 6\zeta_1 + \zeta_2 + \zeta_3\right). \end{aligned} \tag{75}$$

Here $p_{\eta_j} = Q_j \dot{\eta}_j$ and $p_{\zeta_j} = Q'_j \dot{\zeta}_j$. The coefficients f and 6 count the atomic and cell degrees of freedom coupled to each first thermostat; each later thermostat controls one preceding chain variable. The PV term accounts for work against the imposed pressure. Under the constant hydrostatic target, this expression agrees with the LAMMPS reservoir-energy terms up to an additive constant [13]. It is conserved by the continuous extended equations, while numerical integration introduces a timestep-dependent error. The cell and reservoir momenta are extended dynamical variables; ordinary Hamilton equations applied to this expression alone do not reproduce the implemented MTK dynamics.

Production uses LAMMPS `fix npt`: Nosé–Hoover chains of length three thermostat both the atoms and the barostat. Its flexible-cell equations follow Shinoda's formulation combining the MTK hydrostatic equations with Parrinello–Rahman strain dynamics. The `tri` option allows all six cell degrees of freedom to vary independently, with equal normal-pressure targets and zero shear targets; `mtk yes` enables the ensemble-measure correction [12]. The configured response times are $\tau_T = 100\,\mathrm{fs}$ and $\tau_P = 1000\,\mathrm{fs}$, at a target pressure of 1 bar (Section 3.4).

Thermostat and barostat damping times set their response rates. Excessively fast coupling can excite oscillations, while slow coupling delays relaxation. The physical energy E fluctuates as the reservoirs exchange energy and volume with the system; auxiliary reservoir energies are excluded from the physical enthalpy $H = E + PV$.

2.4.4 From initialization to equilibrium averages

The workflow starts from a periodically valid guest-loaded structure. A fixed-cell minimization removes high-energy contacts; thermal velocities are then drawn from the Maxwell–Boltzmann distribution. Minimization finds a local minimum and does not establish equilibrium at finite T and P. An initial trajectory interval is therefore discarded while the cell, framework, and guest distributions relax.

Production frames must show stationary enthalpy and structural observables. A stable kinetic temperature alone does not demonstrate exploration of slow adsorption or framework modes. Independent replicas help expose incomplete sampling, and autocorrelation limits the information in successive frames (Section 2.3). Mean enthalpies at neighboring temperatures then provide the classical heat-capacity derivative. The numerical settings, continuation procedure, and output selection are given in Section 3.4.

2.5 Machine learning and deep learning

2.5.1 Learning from reference data

Machine learning fits a function to examples so that it can predict outputs for new inputs. For interatomic potentials, this is a supervised regression problem: an input $\boldsymbol{x}$ describes an atomic configuration, and the target $\boldsymbol{y}$ contains continuous quantities such as energies, forces, or stresses from electronic-structure calculations. A model f_θ predicts $\hat{\boldsymbol{y}} = f_\theta(\boldsymbol{x})$ using adjustable parameters θ.

A simple example is linear regression, $f_\theta(x) = a_0 + a_1 x$, where the parameters are the intercept and slope. Training selects these parameters by minimizing a loss that measures disagreement with the reference data. For N_{train} examples, a regularized squared-error objective is

$$\mathcal{L}(\theta) = \frac{1}{N_{\mathrm{train}}} \sum_{n=1}^{N_{\mathrm{train}}} \|\hat{\boldsymbol{y}}_n - \boldsymbol{y}_n\|^2 + \lambda \mathcal{R}(\theta), \tag{76}$$

where $\mathcal{R}$ is an optional penalty and λ_{reg} controls its strength. With no penalty and scalar outputs, the objective reduces to the mean squared error. When energies, forces, and stresses are fitted together, their errors require separate weights and consistent units.

2.5.2 Gradient descent

Gradient descent updates the parameters in the direction opposite to the loss gradient:

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k), \tag{77}$$

where k counts updates and η is the learning rate. The gradient gives the direction of steepest local increase in the chosen parameter coordinates, so a sufficiently small step in the opposite direction lowers the loss. A large learning rate can cause overshooting or divergence; a very small one slows convergence. For neural networks, the loss is generally nonconvex, and convergence to a global minimum is not guaranteed. Training commonly estimates each gradient from a small batch of examples rather than the entire dataset.

Figure 2 illustrates the updates on a contour map of a quadratic loss. The axes represent adjustable model parameters, and each contour connects parameter values with equal loss.

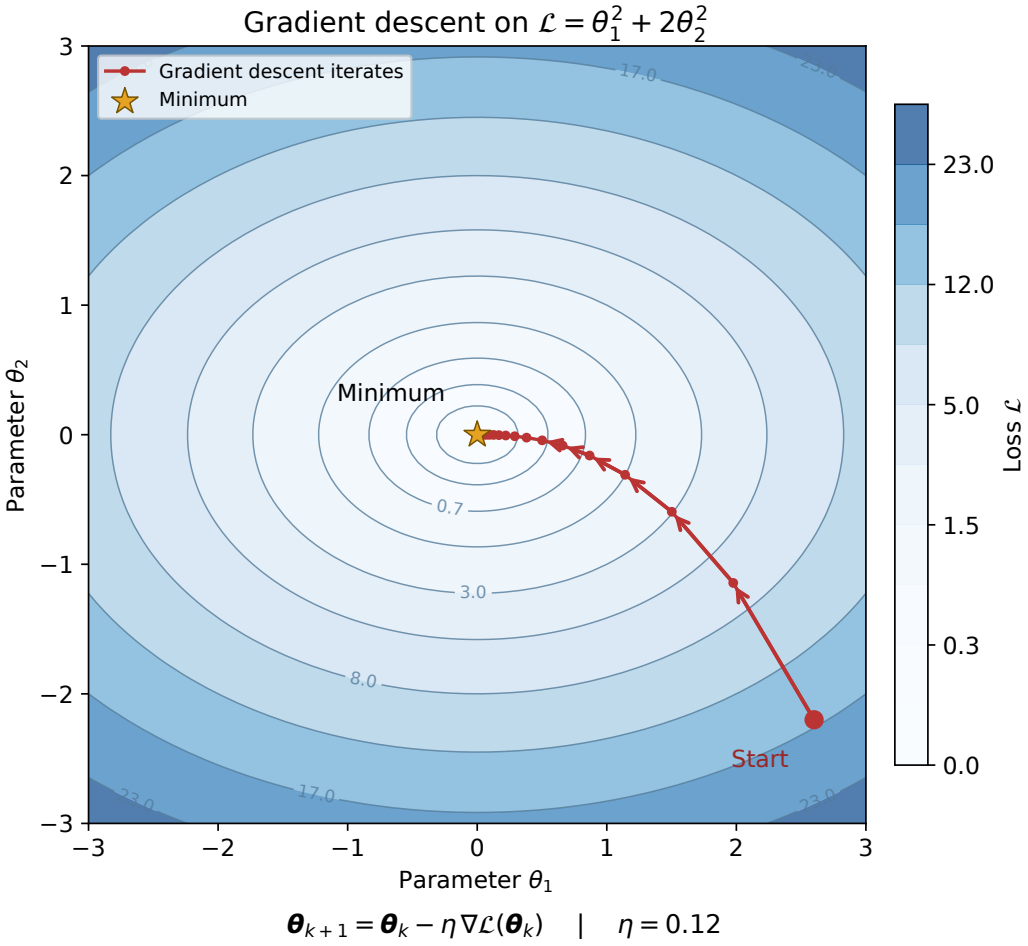


Figure 2: Gradient descent viewed from above on the illustrative loss $\mathcal{L}(\theta_1, \theta_2) = \theta_1^2 + 2\theta_2^2$. Starting from $(2.6, -2.2)$, each update follows $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \mathcal{L}(\boldsymbol{\theta}_k)$ with learning rate $\eta = 0.12$. Red points show twenty updates and their initial point; arrows indicate the direction of the first six steps. The loss decreases toward the minimum at $(0, 0)$, marked by a star. Contour labels and the color scale indicate the loss; darker shading corresponds to larger values. This quadratic example illustrates optimization; neural-network loss surfaces are generally more complex.

2.5.3 Generalization and overfitting

The aim of training is to predict configurations absent from the training set. A model that is too restrictive can underfit the data, while an overly flexible model can fit noise and generalize poorly. Figure 3 compares a linear model with polynomial models $f(x) = \sum_{k=0}^{d} a_k x^k$ fitted to the same synthetic observations. The straight line misses the curvature, the cubic polynomial captures the trend, and the degree-eleven polynomial interpolates the noisy training points but has a larger validation error.

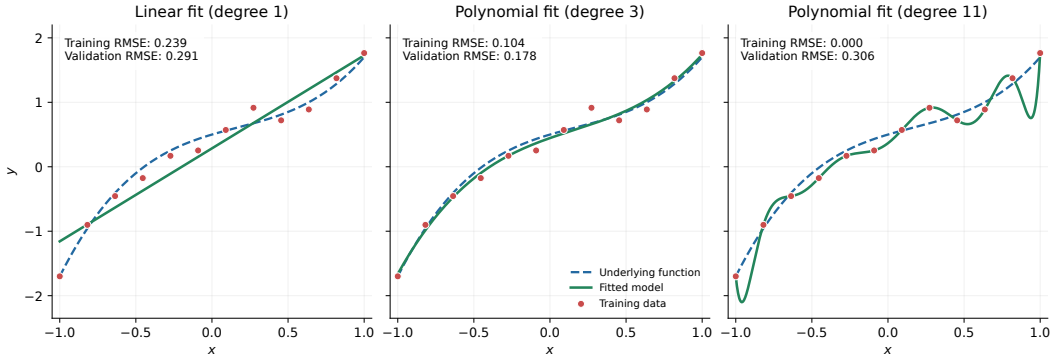


Figure 3: Linear and polynomial regression on twelve synthetic training points sampled from a cubic function with Gaussian noise. All panels use the same data and axis limits. The dashed blue curve is the underlying function, red points are training observations, and the green curve is the fitted model. Root-mean-square errors (RMSE) are calculated on the training set and on 200 independent noisy validation observations, which are omitted for clarity. The degree-eleven model has essentially zero training error but a larger validation error than the cubic model. These are illustrative data, not MLIP predictions.

A separate validation set guides choices such as model size, learning rate, and regularization strength. Monitoring its error also enables early stopping: if training error continues to decrease while validation error rises, an earlier set of parameters may generalize better. A held-out test set assesses the final model after these choices have been made. Correlated trajectory frames should remain in the same split to avoid overly optimistic errors.

Regularization provides another way to limit overfitting. An L_1 penalty encourages sparse parameters, whereas an L_2 penalty discourages large parameter values. Neither validation nor regularization guarantees reliable predictions outside the configurations represented by the reference data.

2.5.4 Neural networks and the forward pass

Deep learning uses neural networks with multiple learned layers to represent complex relationships between inputs and outputs. Neurons carry computed values; the trainable parameters are the connection weights and biases. As shown in Figure 4, each hidden layer forms weighted sums of the previous layer's outputs and applies a nonlinear activation:

$$\begin{aligned} \boldsymbol{z}^{(\ell+1)} &= \boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell)} + \boldsymbol{b}^{(\ell)}, \\ \boldsymbol{h}^{(\ell+1)} &= \sigma\left(\boldsymbol{z}^{(\ell+1)}\right), \qquad \boldsymbol{h}^{(0)} = \boldsymbol{x}. \end{aligned} \tag{78}$$

Here $\boldsymbol{z}$ denotes values before activation and $\boldsymbol{h}$ denotes the resulting features. The activation σ acts componentwise. Without nonlinear activations, composing these layers would remain an affine map. For L hidden layers and scalar regression, a final linear readout gives

$$\hat{y} = \boldsymbol{w}_{\text{out}}^{\mathrm{T}}\boldsymbol{h}^{(L)} + b_{\text{out}}. \tag{79}$$

Evaluating these operations from input to output is the forward pass. The hidden layers build intermediate, or latent, representations rather than requiring every useful feature to be specified in advance. Suitable networks with sufficient capacity can approximate continuous functions on compact domains, but this representational ability alone does not guarantee successful training or generalization.

2.5.5 Backpropagation and parameter updates

Training alternates between evaluating predictions and a loss, computing its parameter gradients, and applying an optimizer update. Backpropagation computes those gradients by applying the chain rule in reverse through the network's operations. Intermediate values from the forward pass are retained or recomputed so that a single reverse pass can accumulate the derivatives with respect to all trainable parameters. Gradient descent then uses these derivatives in Eq. (77).

Appendix C develops the computational-graph view, a worked example, and the layer-by-layer derivation. For interatomic potentials, the same AD machinery differentiates the energy with respect to coordinates to obtain

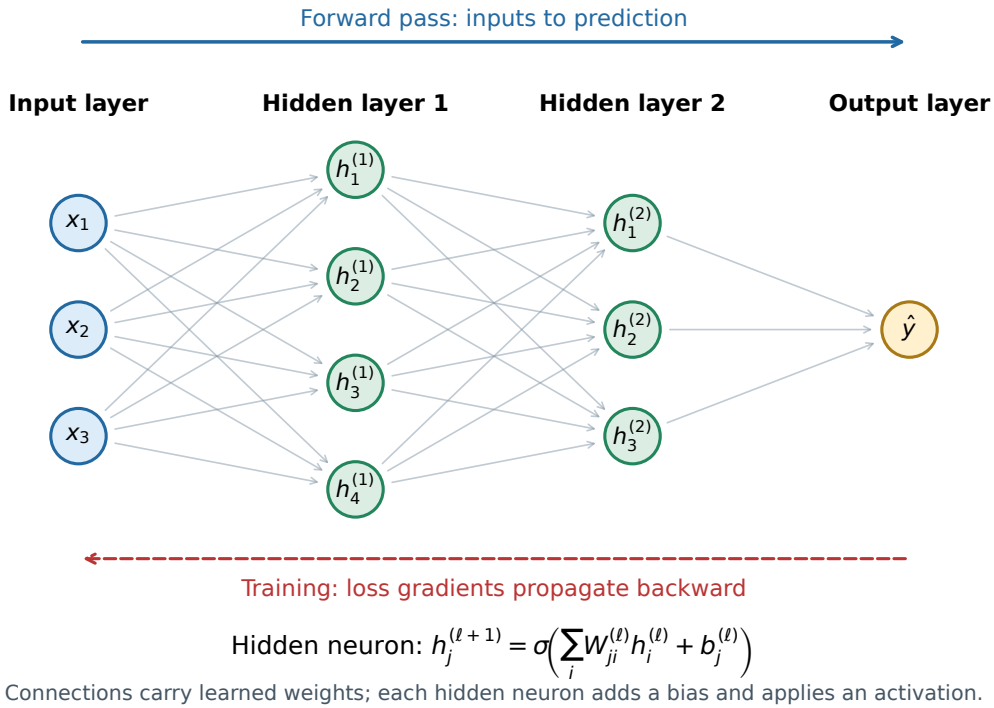


Figure 4: A simple fully connected neural network with three inputs, two hidden layers, and one scalar output $\hat{y}$. Circles represent input features or neurons, and connections represent learned weights. The forward pass evaluates the prediction from left to right. During training, backpropagation carries loss gradients in the reverse direction to calculate parameter updates. Biases are included in the neuron equation but omitted from the connections for clarity. This generic network illustrates the layer operations used within more specialized interatomic-potential architectures.

forces, and differentiates again for Hessian information (Section 2.7.2). This makes smooth activations and cutoff functions relevant to physical derivatives as well as training.

2.6 Machine-learned interatomic potentials

A machine-learned interatomic potential (MLIP) approximates the energy of an atomic configuration using reference electronic-structure data. This reduces the cost of long trajectories and Hessian calculations. Reliable predictions require reference data that cover the configurations visited by the loaded framework. This section describes PET; a step-by-step explanation of its transformer operations is provided in Appendix A. Checkpoint and conversion details belong to Section 3.3.

2.6.1 Point Edge Transformer architecture

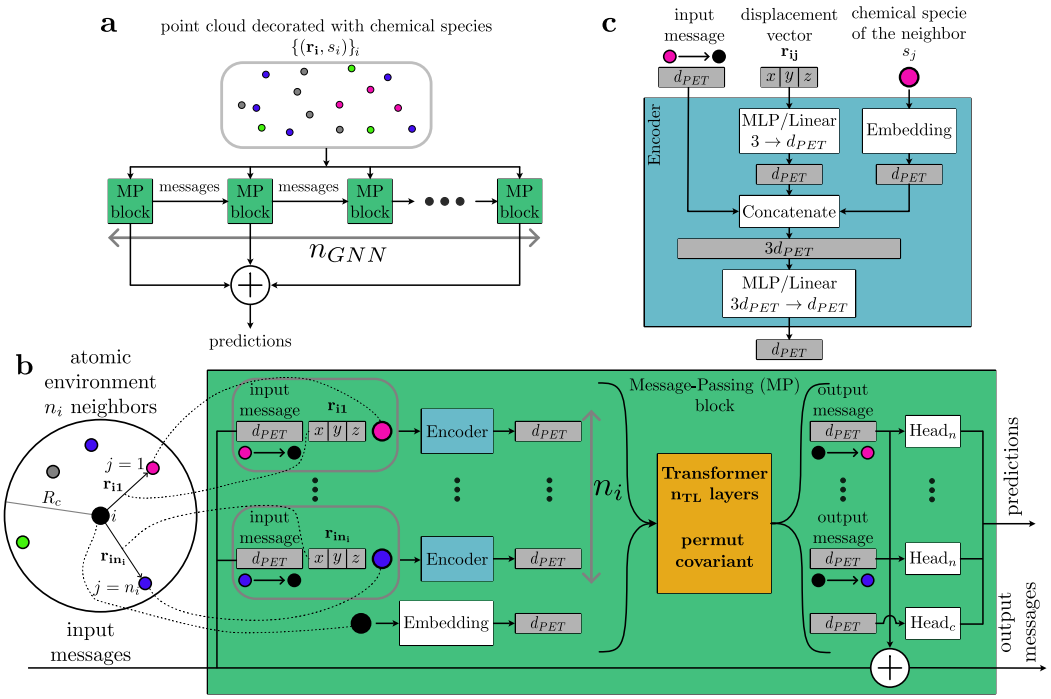


Figure 5: Point Edge Transformer architecture, reproduced from Ref. [25]. Each message-passing block embeds geometry, species, and incoming messages, processes the local tokens with a transformer, and produces both target contributions and outgoing directed messages. Local transformer depth n_{TL} and inter-environment message-passing depth n_{GNN} play different roles.

Point Edge Transformer (PET) computes an energy by processing local atomic geometry, exchanging directed messages between environments, and summing learned contributions. The description below follows Section 6, Figure 2, and Appendix A of Ref. [25], using the paper's notation. The later PET-MAD potential [24] uses this architectural family; the exact checkpoint settings used here are given in Section 3.3.

Model overview The first step in the PET architecture is to define a local environment $A_i = \{(\mathbf{r}_{ij}, s_j) | r_{ij} < R_c\}$ for an atom i, where $\mathbf{r}_{ij}$ represents the position of atom j relative to atom i, s_j is the species of atom j, which is a categorical variable that could correspond to the the atomic number Z_j, and R_c is the cutoff radius, which limits the number of neighbors considered in the environment. The environment includes all neighboring atoms j within this cutoff distance from the central atom i. The same local block is applied to all atomic environments, usually batched for parallel evaluation, and the total energy sums their learned contributions. Once the environment is defined, the PET architecture processes it through n_{GNN} message-passing (MP) blocks as shown in Figure 5 a. The first MP block takes as input only the atomic environment A_i and produces an output (here the energy), as well as an output message. The next MP block takes as input the same atomic environment A_i as well as the output message from the previous block. It will then output a new energy and a new output message. This process is reapeated for all n_{GNN} MP blocks, with each block taking as input the same atomic environment A_i and the output message from the previous block. The final output of the PET architecture is the sum of all energy contributions from each MP block.

Encoding and embedding At each message-passing block k, PET converts the species, geometry, and incoming messages of the environment A_i into tokens of width d_{PET} (Figure 5b–c). Following Appendix A of Ref. [25], the central and neighbor species use separate learned embedding maps, $E_c^{(k)}(s_i), E_n^{(k)}(s_j) \in \mathbb{R}^{d_{\mathrm{PET}}}$. These maps allow the same chemical species to have different representations in the central and neighbor roles. The relative position of neighbor j is embedded through a learned linear layer followed by a SiLU activation:

$$E_r^{(k)}(\boldsymbol{r}_{ij}) = \sigma\Big(L^{(k)}(\boldsymbol{r}_{ij})\Big), \qquad L^{(k)} : \mathbb{R}^3 \longrightarrow \mathbb{R}^{d_{\mathrm{PET}}}. \tag{80}$$

This maps the three Cartesian components to the same feature width as the species embeddings. The resulting features encode spatial geometry and are not required to transform as physical vectors under rotations.

For blocks after the first, the neighbor also supplies the incoming message $\tilde{x}_{ij}^{(k-1)} \in \mathbb{R}^{d_{\mathrm{PET}}}$, sent from j to i in the preceding block. Equation (5) of Ref. [25] combines this message with the species and position embeddings to form the neighbor token:

$$t_j^{(k)} = M^{(k)}\Big[\mathrm{concatenate}\Big(\tilde{x}_{ij}^{(k-1)}, E_n^{(k)}(s_j), E_r^{(k)}(\boldsymbol{r}_{ij})\Big)\Big]. \tag{81}$$

Concatenation produces $3d_{\mathrm{PET}}$ components. The one-hidden-layer MLP $M^{(k)}$ mixes these inputs and compresses them to a token of width d_{PET}. The central atom i is implicit in the notation $t_j^{(k)}$.

In the first block, no incoming messages exist, so the message input is omitted and $M^{(1)}$ maps $2d_{\mathrm{PET}}$ features to d_{PET}. The central token is simply $E_c^{(k)}(s_i)$ and has no displacement input. An environment containing m neighbors therefore supplies $m+1$ tokens to the transformer: one for each neighbor and one for the center.

Appendix A also describes an alternative that initializes messages with species embeddings and omits the separate neighbor-species input, giving $2d_{\mathrm{PET}}$ inputs at every block. This is an alternative to the token construction above, rather than an additional encoding step.

Transformer layers and attention The tokens constructed above are processed by n_{TL} transformer layers. As explained in Appendix A, self-attention lets each token use information from every other token in the environment. Consequently, a neighbor's representation can depend on the surrounding geometry as well as its own displacement from the center.

A neighbor must lose its influence smoothly as it approaches R_c. Otherwise, removing its token at the cutoff would abruptly change the attention weights and their normalization, potentially making the predicted energy discontinuous. PET therefore introduces a smooth cutoff, given in Appendix F.1, Eq. (9), of Ref. [25]:

$$f_c(r \mid R_c, \Delta_{R_c}) = \begin{cases} 1, & r \le R_c - \Delta_{R_c}, \\ \frac{1}{2}\left[\tanh\left(\frac{1}{x+1} + \frac{1}{x-1}\right) + 1\right], & R_c - \Delta_{R_c} < r < R_c, \\ 0, & r \ge R_c, \end{cases} \tag{82}$$

$$x = 2\frac{r - R_c + 0.5\Delta_{R_c}}{\Delta_{R_c}} \quad \text{inside the switching interval.}$$

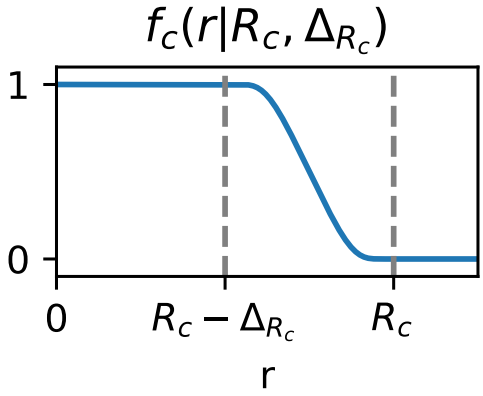


Figure 6: Smooth cutoff function $f_c(r \mid R_c, \Delta_{R_c})$ from Eq. (82) with $R_c = 5.0\,\text{Å}$ and $\Delta_{R_c} = 0.5\,\text{Å}$. The function is unity for $r < R_c - \Delta_{R_c}$, smoothly decreases to zero over the switching interval, and is zero for $r > R_c$.

The width $\Delta_{R_c} > 0$ controls the interval over which a neighbor is switched off. The constant branches are used at the endpoints, where the fractions in the middle branch are undefined. The function joins smoothly to these branches, although a narrow switching interval can still produce large derivatives within it.

Section 6 of Ref. [25] applies this cutoff to the ordinary softmax attention weights and then renormalizes them. Let a, b, c index tokens in the environment centered on atom i, with $c_b = f_c(r_{ib} \mid R_c, \Delta_{R_c})$ for a neighbor and $c_b = 1$ for the central token. For one attention head, the modified weights and output are

$$\alpha_{ab}^{\text{cut}} = \frac{\alpha_{ab} c_b}{\sum_c \alpha_{ac} c_c}, \qquad \boldsymbol{y}_a = \sum_b \alpha_{ab}^{\text{cut}} \boldsymbol{v}_b, \tag{83}$$

where α_{ab} are the ordinary weights from Eq. (108) and $\boldsymbol{v}_b$ is the value vector. The superscript cut distinguishes the modified weights from the original ones. As a neighbor leaves, it disappears smoothly from both the weighted sum and its normalization. The central token retains unit cutoff weight, keeping the denominator positive in exact arithmetic.

Each transformer layer combines multi-head attention with token-wise feedforward operations, residual connections, and layer normalization; Eq. (114) illustrates the general arrangement. The output tokens of one layer become the inputs to the next, adding processing within the same environment. Shared operations preserve permutation equivariance: reordering the neighbor tokens produces the same reordering of their outputs, so their association with individual neighbors is retained (Figure 5b).

Transformer outputs The n_{TL} transformer layers produce $n_{\text{i+1}}$ outputs, $x_i^{(k)}$ for the central token and $x_{ji}^{(k)}$ for each neighbor j. The $x_{ji}^{(k)}$ correspond to the output messages that will be fed into the next MP block. The energy prediction of the block is computed using $H_n^{(k)}(x_{ji}^{(k)})$ for the neighbor contributions and $H_c^{(k)}(x_i^{(k)})$ for the central contribution, which are learned readouts that output scalar values. Using the cutoff function $f_c(r_{ij} \mid R_c, \Delta_{R_c})$ to smoothly remove contributions from neighbors beyond the cutoff distance, the neighbor contributions are summed to produce the total energy contribution from the block:

$$y_i^{\text{bond}\,(k)} = \sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c}). \tag{84}$$

The cutoff is required here as well as inside attention. Attention removes a neighbor's influence on surviving tokens, but its own output need not vanish; the readout cutoff removes its direct contribution to the energy. The alternative in the paper's Eq. (6) is

$$y_i^{\text{bond}\,(k)} = \frac{\sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c})}{\sum_{j \in A_i} f_c(r_{ij} \mid R_c, \Delta_{R_c})}. \tag{85}$$

This weighted average changes sensitivity to coordination and requires special handling when all neighbor weights vanish. The sum naturally returns zero for an empty neighbor list. The two readouts are alternatives that must be selected consistently in training and inference.

Appendix B adds fitted atomic self-contributions to the network output. Writing $E_{\text{self}} = \sum_i b_{s_i}$ for that composition-dependent baseline, the following expression assembles the paper's central, edge, and baseline terms:

$$\tilde{E} = E_{\text{self}} + \sum_{k=1}^{n_{\text{GNN}}} \sum_i \left[H_c^{(k)}(x_i^{(k)}) + y_i^{\text{bond}\,(k)} \right]. \tag{86}$$

Equation (86) is bookkeeping assembled from the text, not a separately numbered equation in Ref. [25]; E_{self} and b_s are explanatory symbols. The total $\tilde{E}$ is the learned potential U_θ used elsewhere in this report. Atomic self-contributions are fitted by linear regression on species counts before network training, subtracted from the targets during training, and restored at inference. They have zero Cartesian derivatives at fixed composition.

All blocks contribute to the readout. Central and edge terms are learned parts of the total energy, not uniquely defined physical atomic or bond energies. An edge output depends on its environment and incoming messages, so it represents a many-body contribution; no extra pair-counting factor of one half is implied. Forces follow by differentiating the total energy through all environments, messages, attention normalizations, and cutoffs, as in Eq. (66).

Design implications. With m neighbors, attention forms $(m+1) \times (m+1)$ token-pair weights. Its local cost is quadratic in neighbor count, while total cost remains approximately linear in atom count at bounded neighborhoods and fixed architecture. Increasing the cutoff, feature width, local transformer depth, or message-passing depth changes different parts of this cost and of the accessible information. Section 7 of Ref. [25] illustrates this distinction on water: six message-passing blocks with two transformer layers each outperform twelve blocks with one layer each in that test. This supports tuning the two depths separately, not a universal choice of their values. Cutoff smoothness, symmetry treatment, and the coverage of training data remain separate design requirements.

2.6.2 Training and validity

Appendix B, Eq. (8), of Ref. [25] trains against energies and forces using

$$L = w_E \frac{(\tilde{E} - E)^2}{\mathrm{MSE}_E} + \frac{\frac{1}{3N}\sum_{i\alpha}\left(-\frac{\partial \tilde{E}}{\partial r_{i\alpha}} - F_{i\alpha}\right)^2}{\mathrm{MSE}_F}. \tag{87}$$

Here $\tilde{E}$ and E are the predicted and reference energies of a configuration, $F_{i\alpha}$ are reference force components, and N is its atom count. Training combines such contributions over samples, with consistent energy offsets in predictions and references. The denominators MSE_E and MSE_F are exponential moving averages of validation-set mean squared errors, updated once per epoch. They provide scales for the two residuals, while w_E controls their relative weight. The force error is averaged over $3N$ components. The energy numerator uses total energies in this equation; Appendix C.2 describes a different normalization for the HME21 benchmark. Force training differentiates the energy gradient with respect to the learned model parameters.

The original training procedure uses random rotations at each epoch, Adam optimization, and validation-based early stopping. Positions and force labels must be rotated together, along with the cell for periodic samples. Appendix B's defaults are $d_{\mathrm{PET}} = 128$, $n_{\mathrm{GNN}} = n_{\mathrm{TL}} = 3$, four attention heads, and transformer feed-forward width 512, with dataset-specific exceptions. It reports no dropout or weight decay for those experiments. These are the original paper's settings, rather than a prescription for the later checkpoints. This project uses pretrained PET models; PET-MAD was trained on a chemically diverse DFT dataset [24].

An MLIP inherits its reference method's approximations and may extrapolate poorly to unfamiliar environments. Hold-out errors, trajectory stability, and uncertainty indicators test different aspects of validity. Curvatures need explicit validation because the usual loss does not constrain Hessians directly; PET's methane benchmarks do not establish accuracy for adsorption in MOF-5.

2.7 Harmonic force constants and automatic differentiation

Let $\boldsymbol{R}_0$ be a stationary reference structure and $\boldsymbol{u} = \boldsymbol{R} - \boldsymbol{R}_0$. Expanding the potential energy gives

$$\begin{aligned} U(\boldsymbol{R}_0 + \boldsymbol{u}) = U(\boldsymbol{R}_0) + \underbrace{\nabla U(\boldsymbol{R}_0) \cdot \boldsymbol{u}}_{=0 \text{ at a minimum}} \\ + \frac{1}{2}\boldsymbol{u}^{\mathrm{T}} \boldsymbol{H} \boldsymbol{u} + \mathcal{O}(u^3), \end{aligned} \tag{88}$$

with Cartesian Hessian

$$H_{i\alpha,j\beta} = \frac{\partial^2 U}{\partial R_{i\alpha} \partial R_{j\beta}} = -\frac{\partial F_{i\alpha}}{\partial R_{j\beta}}. \tag{89}$$

2.7.1 Normal modes and translational invariance

For N atoms, $\boldsymbol{H}$ has dimensions $3N \times 3N$. Mass weighting connects its curvature to the normal-mode frequencies used in Section 2.3.3:

$$D_{i\alpha,j\beta} = \frac{H_{i\alpha,j\beta}}{\sqrt{m_i m_j}}, \qquad \boldsymbol{D}\boldsymbol{e}_k = \omega_k^2 \boldsymbol{e}_k. \tag{90}$$

Positive eigenvalues describe stable vibrations; negative eigenvalues indicate unstable directions and give imaginary frequencies. The Hessian symbol $\boldsymbol{H}$ denotes a matrix, distinct from the scalar enthalpy H.

Translational invariance requires a uniform displacement of all atoms to leave the energy unchanged. The corresponding acoustic sum rule is

$$\sum_j H_{i\alpha,j\beta} = 0. \tag{91}$$

The three uniform translations therefore have zero frequency. At a stable minimum, the remaining eigenvalues should be positive. Numerical symmetrization and an acoustic-sum-rule correction can reduce violations of Hessian symmetry and translational invariance, but they do not turn an unstable stationary structure into a minimum. Negative eigenvalues must remain identifiable when assessing a reference spectrum. The positive frequencies enter the oscillator heat capacities derived in Section 2.3.3.

2.7.2 Automatic differentiation

Appendix C develops reverse-mode AD for parameter gradients during training. Here the parameters are fixed, and the same chain-rule machinery differentiates the energy with respect to atomic coordinates. A neural-network energy composes differentiable operations:

$$\boldsymbol{z}^{(0)} = \boldsymbol{R}, \qquad \boldsymbol{z}^{(\ell+1)} = \phi_\ell(\boldsymbol{z}^{(\ell)}; \theta), \qquad U_\theta = \rho(\boldsymbol{z}^{(L)}). \tag{92}$$

AD applies their chain rules without a finite-displacement step [32]. It differentiates the implemented potential, up to floating-point error; it does not improve the potential's accuracy. For $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$,

$$(\boldsymbol{J}_f)_{pq} = \frac{\partial f_p}{\partial x_q}, \qquad \boldsymbol{J}_f \in \mathbb{R}^{m\times n}. \tag{93}$$

Forward AD evaluates $\boldsymbol{J}_f \boldsymbol{v}$ (JVP); reverse AD evaluates $\boldsymbol{w}^{\mathrm{T}} \boldsymbol{J}_f$ (VJP). Dense recovery requires n forward or m reverse seeds, without storing layer Jacobians [33]. A single reverse sweep yields ∇U and hence all forces. Forward-over-reverse evaluates

$$\boldsymbol{H}\boldsymbol{v} = \frac{\mathrm{d}}{\mathrm{d}\epsilon} \nabla_{\boldsymbol{R}} U_\theta(\boldsymbol{R} + \epsilon \boldsymbol{v}) \Big|_{\epsilon=0}. \tag{94}$$

Dense Hessian recovery needs $3N$ such products. Appendix B develops the sparse reconstruction used here, including graph coloring and the SADMOF implementation. Cutoff smoothness and neighbor-selection conventions still control which function is differentiated.

3 Methods

The calculation follows the sequence in Figure 7: prepare a loaded periodic structure, sample its classical NPT enthalpy, and evaluate a harmonic correction from a relaxed loaded configuration. The two contributions use the same PET model and composition and are combined according to Eq. (59). This section specifies the systems, numerical settings, and analysis choices used for the reported calculations.

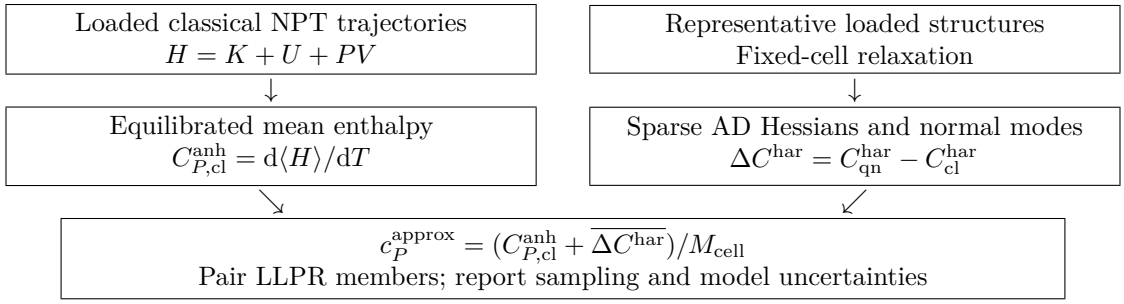
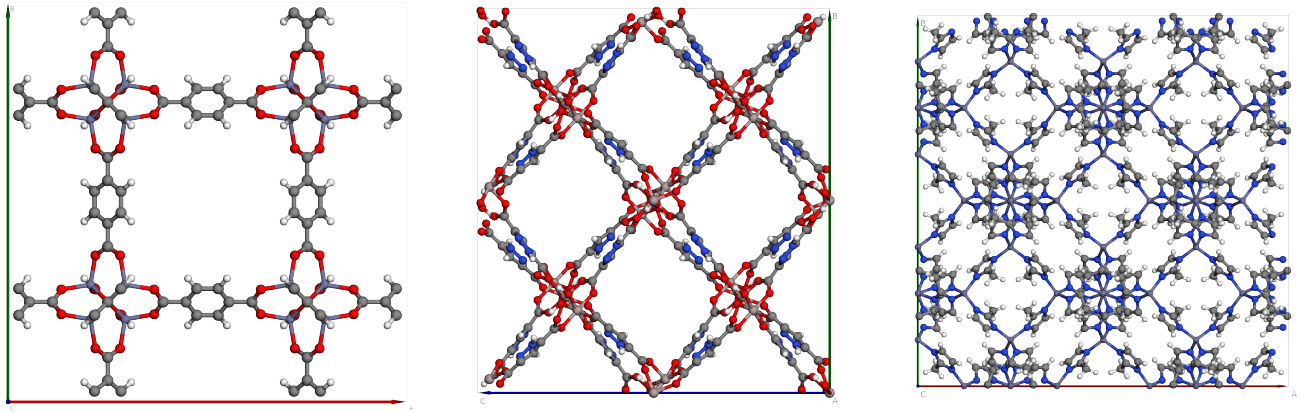


Figure 7: Loaded-system workflow. Accept harmonic references only after force and signed-spectrum checks. The empty-MOF Hessian is evaluated separately, without an empty-MOF MD campaign.

3.1 Frameworks and simulation systems

The reported campaign uses the conventional cubic MOF-5 cell, with initial lattice parameter 25.866 Å and composition $C_{192}H_{96}O_{104}Zn_{32}$ (424 host atoms). Its Zn_4O nodes are connected by benzene-1,4-dicarboxylate linkers. Adding 50, 100, or 150 methane molecules gives 674, 924, or 1174 atoms, respectively. Each loading is simulated with PET-MAD and PET-SOL at eleven temperatures, with one trajectory per model, loading, and temperature, giving 66 trajectories in total. Periodic boundary conditions apply in all three directions. All host and guest atoms move freely; no rigid-molecule or fixed-framework constraints are imposed.

The additional starting structures in Figure 8 are prepared for extensions of this comparison. MOF-303, Al(OH)(PZDC), contains aluminium–oxygen rods connected by 1H-pyrazole-3,5-dicarboxylate linkers [8]; its $2 \times 2 \times 2$ host supercell contains 1024 atoms. ZIF-8, $Zn(mIm)_2$, contains zinc centres joined by 2-methylimidazolate linkers in a sodalite-type network [9]; its $2 \times 2 \times 2$ host supercell contains 2208 atoms. These structures specify the proposed host cells, while the numerical results and run-specific settings below refer to MOF-5/methane.



(a) MOF-5 (b) MOF-303 (c) ZIF-8

Figure 8: Empty framework starting structures rendered with Chemiscope from the project's structure dataset. MOF-5 contains 424 atoms; the MOF-303 and ZIF-8 $2\times2\times2$ supercells contain 1024 and 2208 atoms, respectively. Cell outlines indicate the periodic simulation boxes. Views are independently scaled to reveal the pore geometry.

3.2 Structure preparation and initialization

The MOF-5 calculations use `input/mof5.pdb` for the host and `input/ch4.gro` for the methane template. The preparation routine reads the periodic host and single-molecule template. Randomly oriented guest molecules are placed at fractional positions in the cell, rejecting overlaps with the host or previously placed guests under periodic boundary conditions. The current routine uses a minimum intermolecular atom–atom separation of 1.5 Å and up to 20000 placement attempts per molecule. If sequential insertion stalls, automatic repacking allows up to

20 rounds, removing a random quarter of the inserted molecules between rounds and restarting the full packing every fourth round. Guest geometry is held fixed only during insertion. The original MOF-5/methane structure archives do not retain the insertion-budget metadata, so these current preparation settings cannot be verified retrospectively for those structures.

The campaign seed convention is $20250000 + 100T + r$, where T is the numerical temperature in kelvin and r is the replica index. The reported runs use $r = 1$, giving velocity seeds from 20267501 at 175 K to 20292501 at 425 K. Prepared structures are shared between PET-MAD and PET-SOL at each loading and temperature. Velocities are drawn from a Gaussian distribution at the target temperature, with initial total linear and angular momentum removed. Each temperature is initialized separately.

The archived inputs for the 66 reported MD trajectories contain no pre-MD minimization command. The current preparation workflow adds a fixed-cell LAMMPS FIRE relaxation before assigning velocities, with a maximum Cartesian force-component tolerance of 0.05 eV/Å, zero energy-change tolerance, limits of 2000 iterations and 20000 force evaluations, and maximum displacement 0.05 Å. Failure to meet the force criterion stops a fresh run; the MD timestep is restored after minimization. These settings describe the updated initialization procedure, distinct from the archived MD inputs and from the post-MD relaxations used to obtain Hessians.

3.3 Interatomic potentials and implementations

Two pretrained potentials are used: PET-MAD (`pet-mad-1.5-s-40nn`) and PET-SOL (`pet-sol-s-best`). Their local PET-JAX metadata records the common architecture in Table 1; their weights differ. The architecture and its operations are described in Section 2.6.1.

Table 1: Architecture settings recorded in the local PET-MAD and PET-SOL PET-JAX metadata.

Setting	Value
R_c	8.0 Å
ΔR_c	0.5 Å
Adaptive neighbor-selection setting	40
n_{GNN}	3
n_{TL}	1
d_{PET}	256
Attention heads	8
Transformer feed-forward width	512
Node feature width, `d_node`	1024
Readout feature width, `d_head`	256
Species-embedding capacity	102

LAMMPS evaluates the exported metatomic model on a GPU for MD, while PET-JAX/SADMOF evaluates the corresponding coordinate Hessians. The PET-MAD checkpoint and export have basename `pet-mad-1.5-s_40nn_nostress`, with the PET-JAX conversion `pet-mad-1.5-s_40nn_jax`; the PET-SOL basename is `pet_sol-s-best_nostress`, with `_jax` appended for the conversion. The exports are generated with `upet.save_upet`; conversion preserves the pretrained weights and does not retrain the model. The local PyTorch implementation uses `metatrain` 2026.3.1, and setup pins PET-JAX to commit `ed0add4`.

The generated configurations mark the models as stress-validated. Quantitative use nevertheless requires energy, force, and stress checks on representative loaded structures, together with consistent energy scaling and adaptive-cutoff derivative conventions across the two implementations (Appendix B.9). A configuration flag alone does not establish agreement. All terms of each hybrid curve use its selected model; the two potentials are analyzed separately.

3.4 Classical NPT simulations and enthalpy analysis

The archived LAMMPS inputs use `units metal` and `atom_style atomic`. A Nosé–Hoover-chain thermostat and MTTK barostat propagate a fully flexible triclinic cell. The `tri` pressure setting targets 1 bar for each normal component and zero shear stress, allowing both cell lengths and tilts to vary. Table 2 lists the common parameters for all 66 trajectories.

Thermodynamic logs contain potential and kinetic energies, temperature, volume, the pressure tensor, and cell dimensions. Trajectory records contain atom identities, unwrapped positions, velocities, and forces. Analysis retains steps 200000–1000000, inclusive, corresponding to 8001 samples at the recorded output interval. Logs and configurations are matched by timestep. Restart continuations restore the saved state and run to the original final-step target without assigning new velocities; they form part of the same trajectory.

For each state point, the physical enthalpy is calculated from kinetic and potential energies plus the volume multiplied by the imposed 1 bar pressure. Thermostat and barostat energies are excluded. Mean enthalpies are

Table 2: Parameters shared by the reported MOF-5/methane NPT runs. The discarded interval is an analysis choice, not proof of equilibration.

Parameter	Value
Models	PET-MAD and PET-SOL
Methane molecules per cell	50, 100, 150
Temperatures	175–425 K, in 25 K increments
Replicas per model, loading, and temperature	1
Normal pressure / shear stress targets	1 bar / 0
Particle / barostat thermostat chain lengths	3 / 3
Thermostat / barostat damping times	100 / 1000 fs
Martyna–Tobias–Klein correction	Enabled (`mtk yes`)
Timestep / total steps	0.5 fs / 10^6
Total / discarded / retained duration	500 / 100 / 400 ps
Thermodynamic and trajectory output interval	100 steps (0.05 ps)
Restart interval	10^5 steps (50 ps)
Neighbor-list skin	2 Å
Neighbor rebuild checks	Every step, no delay
Neighbor allocation (`one` / `page`)	10000 / 1000000
Atomic masses: C, H, O, Zn	12.011, 1.008, 15.999, 65.38 u

differentiated using Eq. (43) with $\Delta T = 25\,\mathrm{K}$, so each derivative uses two trajectories separated by 50 K. The extra 175 K and 425 K runs support centered derivatives at the endpoints of the reported 200–400 K interval. Energy, volume, temperature, and structural diagnostics are examined over the retained segment; their convergence is assessed in Section 4.2.

3.5 Harmonic references and heat-capacity assembly

For each model and loading, the final configuration of the 400 K trajectory is relaxed with ASE FIRE using the matching metatomic potential. All atomic coordinates are optimized while the final MD cell is held fixed. The resulting single structure supplies the loaded Hessian used at every reporting temperature. The empty MOF-5 input is relaxed separately at its original cell with each model to provide a harmonic reference; no empty-MOF MD trajectory is used.

Table 3 records the force targets and completed optimization steps for these references. The stopping criterion is the largest atomic force norm, which differs from the Cartesian-component criterion used in the current pre-MD initialization. The four unconverged loaded relaxations reached their recorded step limits. The archives for converged runs retain the completed step count but not the requested maximum budget; these values should therefore not be interpreted as iteration caps. FIRE uses the ASE defaults for its remaining optimizer controls, which are not separately recorded in the relaxation metadata.

Table 3: Archived fixed-cell ASE FIRE settings and completed steps for the reference structures. Loaded references originate at 400 K; zero loading denotes the supplied empty host.

Model	N_{CH_4}	Force target (eV/Å)	Steps	Converged
PET-MAD	0	0.002	70	yes
PET-MAD	50	0.002	10000	no
PET-MAD	100	0.010	1495	yes
PET-MAD	150	0.002	20000	no
PET-SOL	0	0.002	66	yes
PET-SOL	50	0.002	10000	no
PET-SOL	100	0.010	2154	yes
PET-SOL	150	0.002	20000	no

PET-JAX/SADMOF computes the Hessians with the parameters in Table 4. The theoretical construction of normal modes is given in Section 2.7.1, the oscillator heat capacity in Section 2.3.3, and the sparse AD algorithm in Appendix B. The present calculations symmetrize the Hessian, apply the force-constant row-sum acoustic treatment, and diagonalize the mass-weighted matrix while preserving the sign of unstable eigenvalues.

Strict assembly rejects unexpected imaginary modes and more than three near-zero modes. The reported exploratory curves instead retain only frequencies above $1\,\mathrm{cm}^{-1}$, using the same mask for the quantum and classical harmonic terms. This choice permits evaluation of the correction but does not establish stability of the reference structures; the corresponding diagnostics are reported in Section 4.5.

At each reporting temperature, the retained loaded spectrum supplies the quantum-minus-classical harmonic

Table 4: Hessian and heat-capacity settings for the reported analysis. The archived Hessians use float64, overriding the float32 TOML template.

Parameter	Value
Backend / precision	PET-JAX/SADMOF / float64
Structures per model and loading	1 (optimized frame index 0)
Sparse-pattern graph hops	3
AD chunk size	1
Rematerialization / shadow derivatives	Enabled / disabled
Acoustic treatment	Force-constant row sum
Frequency convention	Signed
Near-zero threshold	$\lvert\widetilde{\nu}\rvert \leq 1\,\mathrm{cm}^{-1}$
Imaginary-mode threshold	$\widetilde{\nu} < -1\,\mathrm{cm}^{-1}$
Strict allowance for near-zero modes	At most 3
Standalone harmonic temperature grid	100–500 K, every 10 K
Hybrid reporting grid	200–400 K, every 25 K

correction in Eq. (59). It is added to the classical NPT heat capacity and divided by the total host-plus-guest mass to obtain J/g/K. The empty-framework harmonic curve is normalized by the host mass and shown as a separate reference. Sampling and model uncertainties are treated in the following subsection.

3.6 Uncertainty quantification

3.6.1 LLPR and CEA

When using a machine learning model to predict quantities, there are many sources of uncertainty. The model may not have been trained on data that is representative of the system being studied. It may not capture all relevant physics, which depends on the first principles methods used when doing DFT calculations. If we wanted to have better uncertainty estimates, we would need to have the model trained on different datasets and use these models to predict the same quantity. We could then have a better idea on the uncertainty of the model predictions. Additionaly, we could also build complete different models with different architectures and train them with different datasets. Combining the results from each model to have a final prediction and uncertainty estimate is what we call ensemble learning. In traditional machine learning, ensemble learning is a technique that combines the predictions of multiple models to improve overall performance and provide more robust uncertainty estimates. In practice, ensemble learning is widely used in various machine learning tasks, including regression, classification, and forecasting. One of the most efficient models are tree-based models, such as Random Forests and Gradient Boosting Machines, which combine the predictions of multiple decision trees to improve accuracy and reduce overfitting. However, training multiple models, especially in the case of MLIPs, is an expensive task. Instead of training multiple models, we can use the last-layer prediction rigidity (LLPR) [29] method to estimate the uncertainty of a single model. The idea is that instead of training multiple models, we take the model we are using for the simulation and we create new models by just changing its last layer. The models with the different last layers could be considered as different models that were trained on different datasets. This method is cheap and efficient, and it has been shown to provide good uncertainty estimates for MLIPs. For central energy $\bar{U} = \boldsymbol{w}_0^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x})$ andcovariance-set features $\boldsymbol{F}$,

$$\begin{aligned} \sigma^2_{\mathrm{LLPR}}(\boldsymbol{x}) = \alpha^2 \boldsymbol{f}(\boldsymbol{x})^{\mathrm{T}} \\ \times \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{f}(\boldsymbol{x}). \end{aligned} \tag{95}$$

The regularizer λ stabilizes weakly constrained directions; α is calibrated against energy residuals on a disjoint reference-labeled set using the base potential's electronic-structure definition. Predictions cannot calibrate their own error.

The exported `energy_uncertainty` is an energy standard deviation. Property propagation instead needs persistent, signed energy variations: Each new model is generated by sampling the last-layer weight vector $\boldsymbol{w}_i$ from the following distribution

$$\boldsymbol{w}_i \sim \mathcal{N}\left[\boldsymbol{w}_0, \alpha^2 \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1}\right], \tag{96}$$

The model is then created by multiplying the sampled weight vector with the weights from all the previous layers of the model, which are kept frozen.

$$U^{(i)}(\boldsymbol{x}) = \boldsymbol{w}_i^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x}) \tag{97}$$

The advantage of this method is that it is not only cheap to generate the models, but also we do not need to rerun the simulations with them. The model used to run the simulation can be considered to be the central model and the mean of all the other models generated by sampling the last layer is equal to the central model. That means that we can use the same trajectory generated by the central model to evaluate the quantities with the other models, which will be treated as uncertainties. We can then compute the difference between the potential of the members and the central model as well as the enthalpy of each member.

$$\Delta U_t^{(i)}(T) = U_t^{(i)}(T) - \bar{U}_t(T), \tag{98}$$

$$H_t^{(i)}(T) = K_t + U_t^{(i)}(T) + PV_t, \tag{99}$$

In particular, if we want to calculate the ensemble average of the enthalpy as a function of time, we can use the following equation

$$\left\langle H^{(i)}(T) \right\rangle = \frac{\sum_t e^{-\beta \Delta U_t^{(i)}(T)} H_t^{(i)}}{\sum_t e^{-\beta \Delta U_t^{(i)}(T)}}, \tag{100}$$

It is done using the Boltzmann factors of the energy differences between the central model and the members. When the energy differences $\Delta U_t^{(i)}$ are small, the direct reweighting can be approximated by a first-order cumulant expansion approximation (CEA):

$$\begin{aligned} \left\langle H^{(i)}(T) \right\rangle \approx \left\langle H^{(i)}(T) \right\rangle_{\bar{U}} - \beta \operatorname{cov}_{\bar{U}} \left(H^{(i)}(T), \Delta U^{(i)}(T) \right), \\ \beta = (k_{\mathrm{B}} T)^{-1}. \end{aligned} \tag{101}$$

The CEA is a useful approximation when the energy differences are small, but it is not exact. Direct reweighting remains a diagnostic: $N_{\mathrm{eff}} = 1/\sum_t \widetilde{w}_t^2$ should remain adequate, and $\mathrm{var}_t(\beta \Delta U_t^{(i)}) \ll 1$ is needed for controlled first-order CEA. After having calculated the time-averaged enthalpy for each member, we can then calculate the heat capacity of each member using the following equation

$$C_{P,\mathrm{cl}}^{(i)}(T) = \frac{\mathrm{d}\langle H^{(i)}(T) \rangle_i^{\mathrm{CEA}}}{\mathrm{d}T}, \tag{102}$$

We can then simply calculate the uncertainty at each temperature as the standard deviation of the heat capacities of all the members:

$$\sigma_{C_{P,\mathrm{cl,anh}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{P,\mathrm{cl}}^{(i)}(T) - \overline{C}_{P,\mathrm{cl}}(T) \right]^2 \right\}^{1/2}. \tag{103}$$

This term corresponds to the classical anharmonic heat capacity uncertainty. To compute the uncertainty of the quantum harmonic heat capacity, we need first to select the last frame in the simulation, relax the structure and then compute the Hessian. We then calculate the eigenvalues of the Hessian matrix, which correspond to the vibrational frequencies of the system. We will compute the Hessian for each model and its eigenvalues, and then we will compute the quantum harmonic heat capacity for each model using Eq. (57). Finally, we can compute the uncertainty of the quantum harmonic heat capacity as the standard deviation of the heat capacities of all the models:

$$\sigma_{C_{V,\mathrm{qn,har}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{V,\mathrm{har}}^{(i)}(T) - \overline{C}_{V,\mathrm{har}}(T) \right]^2 \right\}^{1/2}. \tag{104}$$

The total uncertainty of the hybrid heat capacity is then obtained by combining the uncertainties of the classical anharmonic and quantum harmonic heat capacities in quadrature:

$$\sigma_{C_P}(T) = \sqrt{\sigma_{C_{P,\mathrm{cl,anh}}}^2(T) + \sigma_{C_{V,\mathrm{qn,har}}}^2(T)}. \tag{105}$$

4 Results

4.1 Available calculations and status

The campaign contains 66 trajectories: PET-MAD (`pet-mad-1.5-s-40nn`) and PET-SOL (`pet-sol-s-best`), three loadings, eleven temperatures, and one replica per state point. Each 500 ps run contributes 8001 frames after discarding 100 ps: 33 ns simulated and 26.4 ns analyzed in total. Hybrid curves span 200–400 K; 175/425 K support centered endpoint derivatives. Each curve reuses one loaded 400 K reference spectrum.

All estimates are *exploratory*: every loaded spectrum has imaginary modes, and four of six relaxations failed their convergence criterion. Discarding unstable modes does not satisfy the strict acceptance protocol. Bands quantify selected sampling and model errors, excluding unstable-mode bias and unestimated variation across minima.

Trajectory diagnostics come from per-run `summary.json`; central values and relaxation diagnostics come from `heat-capacity.exploratory-discard-imaginary` CSV/JSON archives. Reported bands use refreshed stride-1 evaluation of all 8001 frames with 64 persistent LLPR members, rather than older per-loading uncertainty columns. Figure sources are recorded in `results-provenance.json`.

4.2 Thermodynamic sampling and equilibration

Mean temperatures stay within 0.315 K of target. PET-MAD/100 at 300 K gives 299.854 ± 8.031 K (mean and standard deviation), close to the equipartition width $T\sqrt{2/(3N-3)} = 8.058$ K. This checks kinetic sampling, not configurational ergodicity. Pressure fluctuations span 0.80–1.51 $\times 10^3$ bar with mean errors of 9–18 bar; PET-MAD/100 gives 4.65 ± 13.11 bar, compatible with the 1 bar target. Physical enthalpy always uses imposed pressure.

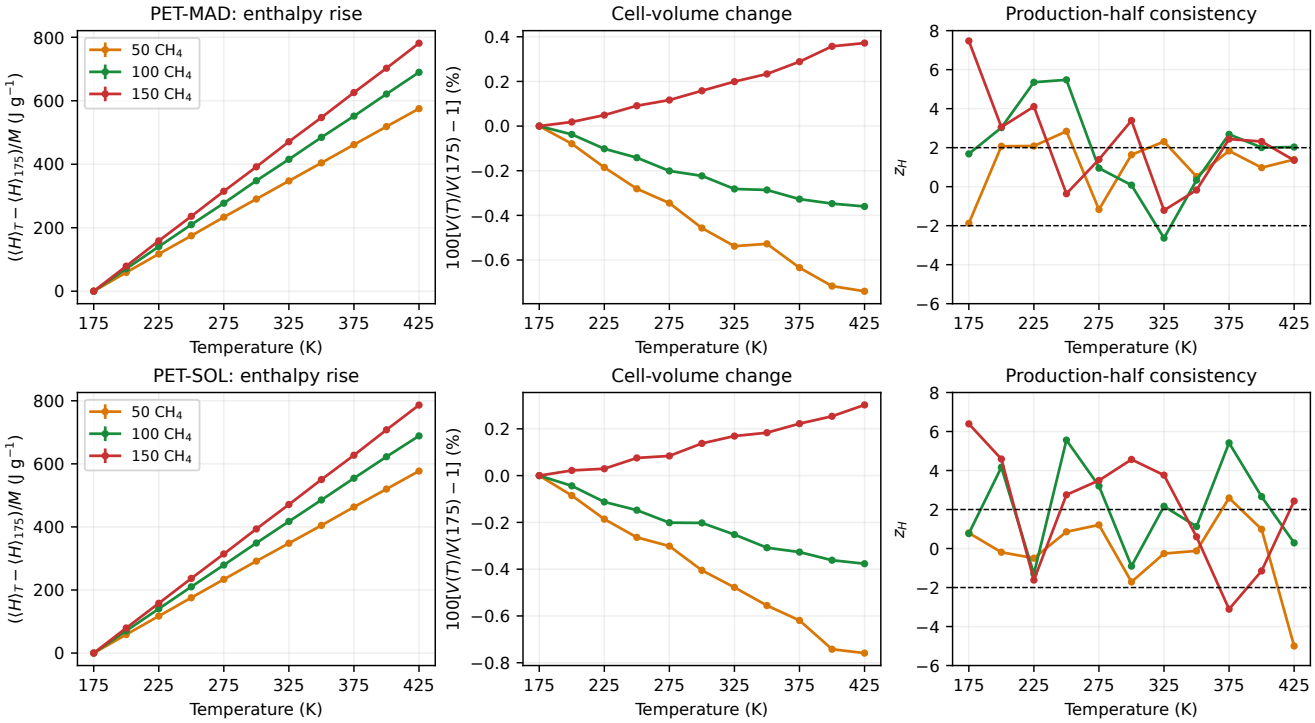


Figure 9: Thermodynamic diagnostics from all 66 trajectory summaries. Left: mass-specific enthalpy increase relative to the same model and loading at 175 K; the baseline is a display offset, and the small bars show the pointwise enthalpy standard errors without propagating the shared baseline error. Middle: fractional change of mean cell volume relative to 175 K. Right: the archived difference between production-half mean enthalpies divided by their combined standard error, z_H; dashed lines mark ± 2. These are diagnostics of the individual runs, not replica averages.

Enthalpy increases smoothly (Figure 9). Archived enthalpy correlation times of 0.095–0.101 ps imply 1980–2115 effective samples and mean errors of 0.015–0.046 eV. These short-time estimates do not exclude slow drift.

Half-trajectory discrepancies flag 33 of 66 runs with $|z_H| > 2$, where

$$z_H = \frac{\overline{H}_{\text{second}} - \overline{H}_{\text{first}}}{\sqrt{\text{SE}^2_{\text{first}} + \text{SE}^2_{\text{second}}}}. \tag{106}$$

Examples are $z_H = 7.47$ for PET-MAD/150 at 175 K and 4.57 for PET-SOL/150 at 300 K. Some RMS-force correlation times reach 26.1 ps, leaving fewer than eight effective samples.

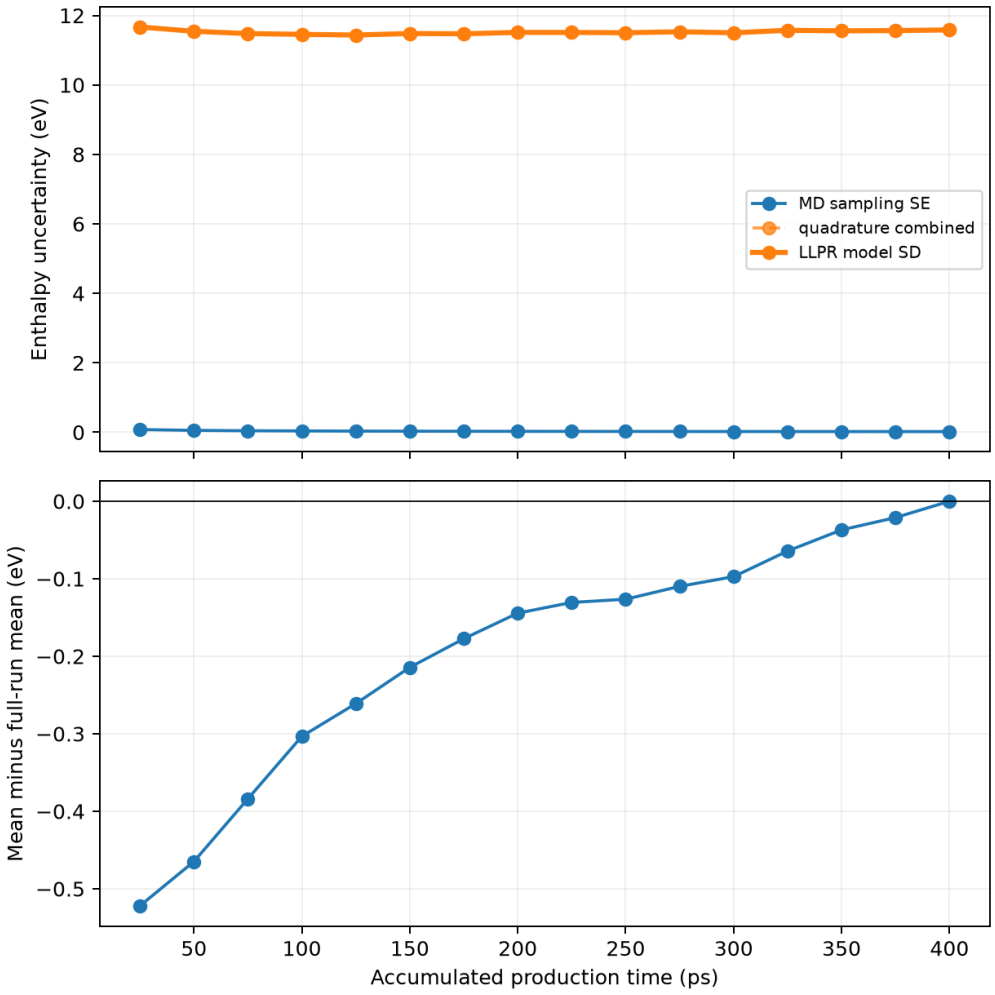


Figure 10: Archived enthalpy convergence diagnostic for PET-MAD with 150 methane molecules at 175 K. The upper panel separates MD sampling error from the LLPR spread of the *enthalpy*, in eV. The lower panel shows the accumulated mean relative to the full-production mean. Its endpoint is zero by construction and is not evidence of equilibration.

In Figure 10, the accumulated mean differs from the final mean by about −0.30 eV after 100 ps and −0.15 eV after 200 ps. A large model spread does not absorb a temperature-dependent sampling bias. Longer independent runs and sensitivity to discarded intervals are needed to validate the reported small MD errors.

4.3 Cell response, framework motion, and methane mobility

Both models predict contraction at 50/100 guests and expansion at 150 over 175–425 K (Figure 9). PET-MAD volumes change from 17146.8 to 17019.9, 17170.2 to 17108.4, and 17210.4 to 17 274.4 Å^3, respectively. Across models the changes are about −0.75%, −0.37%, and +0.3–0.4%. Densities at 300 K are about 0.677, 0.752, and 0.825 g/cm^3, largely reflecting added guest mass. Because $C_P - C_V \propto \alpha_V^2$, contraction does not imply a negative heat-capacity difference; compressibility is also needed to quantify it.

At 300 K, framework RMSDs are 0.659–0.725 Å, while tracked bond-length changes are only 0.055–0.059 Å. Bounded RMSD supports collective motion without gross collapse over the analyzed interval, but tracking a reference neighbor set cannot rule out all bond changes.

Methane 400 ps MSDs at 300 K decrease with loading: 8.9–9.2, 5.1–5.6, and 3.1–3.4 × 10^3 Å^2 for 50, 100, and 150 molecules, respectively. Guest radial distributions peak near 4 Å. Mobility and crowding support the need for anharmonic MD; a single harmonic geometry cannot describe diffusion. Endpoint MSDs alone do not establish diffusion coefficients without a verified diffusive regime, cell-motion control, and replica statistics.

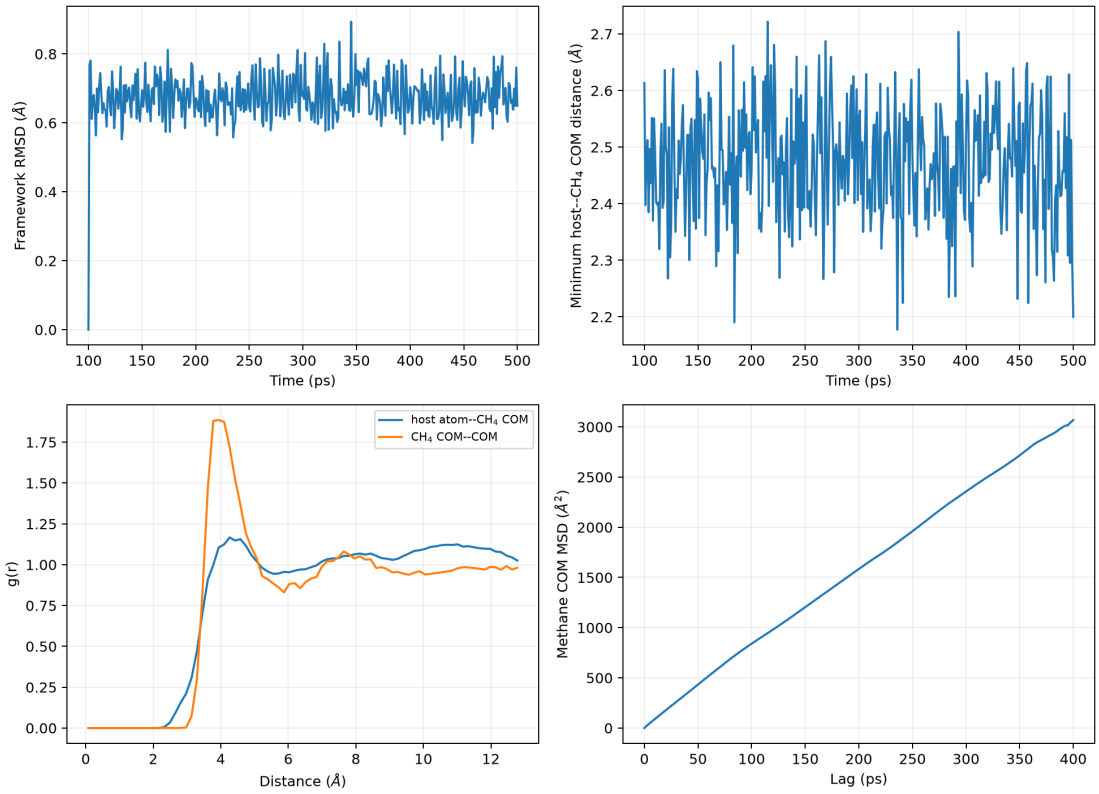


Figure 11: Structural diagnostics for PET-SOL with 150 methane molecules at 300 K: framework RMSD, nearest host-atom–methane center-of-mass distance, radial distributions, and methane center-of-mass mean-squared displacement (MSD). The first analyzed frame at 100 ps defines the framework reference, hence the initial zero RMSD. Long-lag MSD values use fewer time origins than short-lag values.

4.4 Classical, harmonic, and hybrid heat capacities

Table 5 resolves the 300 K contributions. The negative harmonic correction removes 59–63% of the classical term, as quantum occupations suppress high-frequency vibrations. Its increase in magnitude with loading reflects added methane and mixed host–guest modes, requiring loaded rather than empty spectra.

Table 5: Heat capacities and uncertainty estimates at 300 K, all in J/g/K and normalized by the total mass of each loaded cell. c_{cl} is the central enthalpy derivative, Δc_{har} the retained-mode harmonic correction, and c_{hyb} their sum. u_{CEA} and u_{G} use different paired model spreads around the *same* central value and MD standard error σ_{MD}. The final column is the separate variance-based central hybrid estimate. None of these uncertainties includes the bias from discarded imaginary modes or an estimated spread across minima.

Model	$N_{\mathrm{CH_4}}$	c_{cl}	Δc_{har}	c_{hyb}	σ_{MD}	u_{CEA}	u_{G}	$c_{\mathrm{hyb}}^{\mathrm{var}}$
PET-MAD	50	2.281	-1.351	0.929	0.010	0.068	0.127	0.926
PET-MAD	100	2.762	-1.682	1.080	0.010	0.123	0.169	1.044
PET-MAD	150	3.122	-1.957	1.165	0.010	0.113	0.214	1.202
PET-SOL	50	2.289	-1.363	0.926	0.010	0.087	0.150	0.940
PET-SOL	100	2.762	-1.693	1.069	0.010	0.153	0.225	1.024
PET-SOL	150	3.134	-1.969	1.166	0.010	0.103	0.306	1.230

Room-temperature hybrid values rise from approximately 0.93 to 1.07–1.08 and 1.16 J/g/K with loading. Model differences of 0.003, 0.011, and 0.001 J/g/K are below propagated bands and cannot distinguish predictive accuracy.

The curves generally rise over 200–400 K (Figures 12–13). For PET-MAD/50, the classical term decreases from 2.347 to 2.269 J/g/K, while the correction becomes less negative, from -1.551 to $-1.164\,\mathrm{J\,g^{-1}\,K^{-1}}$. Thus the upward hybrid slope need not indicate increasing classical anharmonicity. Adjacent loading bands overlap; significance requires paired propagation of their difference. The grid does not bracket the reference low-temperature minimum, and small local bends remain unresolved within uncertainty.

Empty-framework 300 K values are 0.786 ± 0.021 (PET-MAD) and $0.766 \pm 0.027\,\mathrm{J\,g^{-1}\,K^{-1}}$ (PET-SOL). These are framework-mass-normalized quantum harmonic c_V values with LLPR spread, not hybrid c_P or incremental adsorption heat capacities.

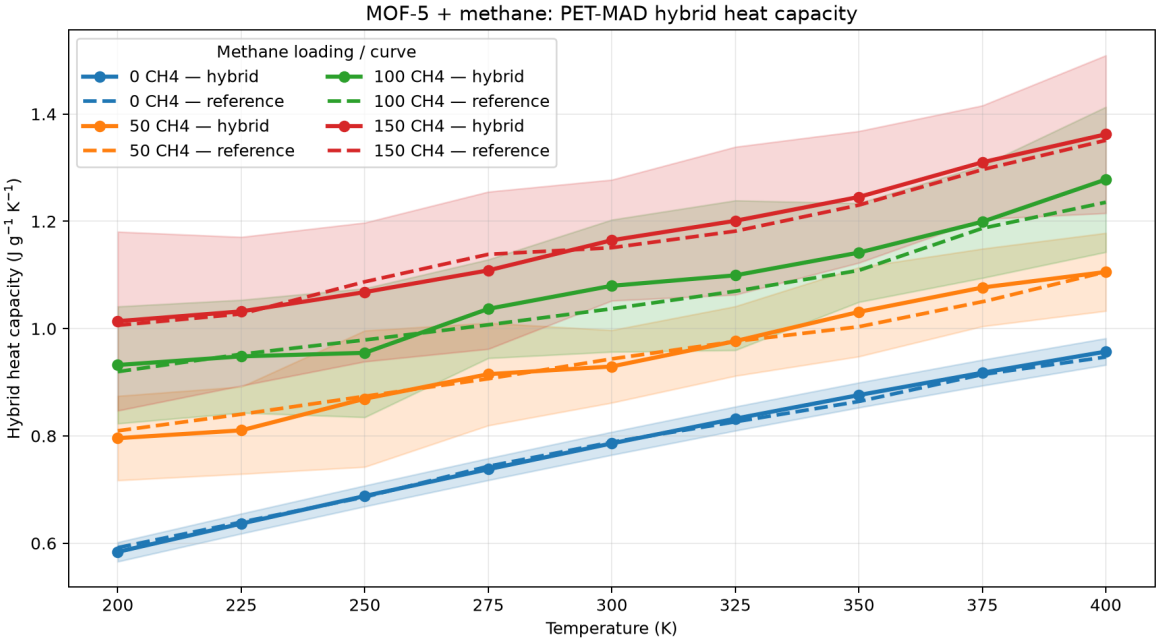


Figure 12: Supplied PET-MAD loading comparison. Loaded solid curves are central classical enthalpy derivatives plus retained-mode harmonic quantum corrections. Their shading is $\pm u_{\mathrm{CEA}}$, combining MD sampling error with correlated classical–harmonic LLPR spread. Despite the common "hybrid" legend, the zero-loading curve is a quantum-harmonic c_V calculation only, with a harmonic LLPR band; no empty-system MD term was computed. Dashed curves are the supplied reference CSVs, whose provenance is discussed in the text.

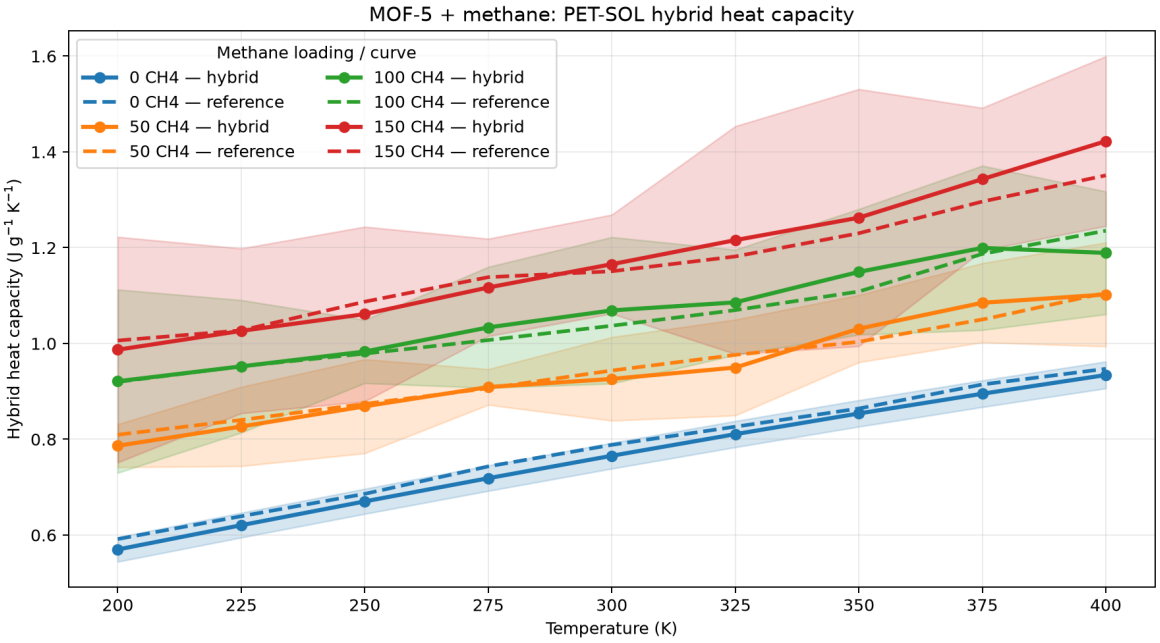


Figure 13: Supplied PET-SOL loading comparison, with the same definitions and exploratory mode treatment as Figure 12. The slight decrease of the 100-molecule central curve at the upper end of the grid is smaller than its uncertainty band and is not evidence of a resolved thermodynamic anomaly. Shading denotes standard uncertainty, not a calibrated confidence interval.

Dashed curves are digitized Kapil *et al.* data [21]. Their 300 K values are approximately 0.788, 0.944, 1.037, and 1.151 J/g/K for 0/50/100/150 guests. Loaded differences from this work are at most 0.043 J/g/K. Agreement is contextual: the digitization has no uncertainty estimates, the force fields and preparation differ, and present sampling/spectra remain unconverged. The earlier approximate empty-host scale is not a separate fitting target.

4.5 Hessian stability

Table 6: Archived central loaded-Hessian diagnostics. All calculations use PET-JAX, float64, three graph hops, and the force-constant row-sum acoustic treatment. "Converged" reproduces the relaxation archive's flag for its requested tolerance. Imaginary modes have signed frequencies below $-1\,\mathrm{cm}^{-1}$; near-zero modes satisfy $|\widetilde{\nu}| \leq 1\,\mathrm{cm}^{-1}$. Retained modes have positive frequencies above the threshold. Signed negative values represent unstable curvature, not physical negative-frequency oscillators.

Model	N_{CH_4}	Converged	F_{max} (eV/Å)	Imaginary	Near zero	Retained	$\widetilde{\nu}_{min}$ (cm^{-1})
PET-MAD	50	no	0.0170	17	5	2000	-53.5
PET-MAD	100	yes	0.0097	114	3	2655	-92.4
PET-MAD	150	no	0.0095	28	3	3491	-66.8
PET-SOL	50	no	0.0054	32	3	1987	-107.7
PET-SOL	100	yes	0.0100	95	3	2674	-138.4
PET-SOL	150	no	0.0049	35	3	3484	-86.5

Force convergence does not establish positive curvature: the two 100-molecule structures pass their archived force criteria but retain 114/95 imaginary modes, down to $-92.4/-138.4\,\mathrm{cm}^{-1}$. Other loaded runs have 17–35 imaginary modes and failed convergence flags; PET-MAD/50 also has five near-zero modes. Run-dependent criteria must be judged from each archive rather than comparing residual forces alone. Empty spectra have four/five imaginary modes, down to $-5.5/-6.6\,\mathrm{cm}^{-1}$.

Discarding negative eigenvalues enables an oscillator sum but does not create a stable minimum. Distinguish insufficient relaxation, unstable stationary structures, and sparse-curvature errors with convergence checks. Matching quantum/classical mode masks avoids inconsistent subtraction; it does not estimate unstable-mode bias. Reusing one 400 K geometry also leaves temperature dependence and minimum-to-minimum variation untested. LLPR spread at a central geometry does not replace structural sampling.

4.6 Uncertainty bands

Figures 14–15 separate MD sampling, classical LLPR, harmonic LLPR, and paired total uncertainty. MD errors are about 0.010 J/g/K at 300 K; model terms dominate. Covariance can make the total smaller than an individual component when the member contributions are anticorrelated.

For PET-SOL with 150 guests at 300 K, classical and harmonic CEA spreads of 0.108 and 0.155 J/g/K combine to 0.103 J/g/K, with inferred correlation −0.75. PET-SOL/100 instead has correlation +0.37 and paired spread 0.153 J/g/K. Independent quadrature would miss both effects; predictive spread is not reduced by $\sqrt{64}$.

Gaussian propagation gives larger 300 K bands in every case (Table 5), with positive classical–harmonic correlations of 0.43–0.63. Smoother or narrower bands do not demonstrate calibration. Recorded $\mathrm{var}(\beta\Delta U_i)$ spans about 2.4 to 1.42×10^3, violating the small-variance CEA criterion; some direct reweighting effective counts approach one. Neither collapsed reweighting nor untested Gaussian tails validate these bands, whose coverage against reference electronic-structure data remains unestablished.

The separate `variance-hybrid` calculation also changes the central classical estimator. At 300 K, hybrid differences from the derivative route range from −0.045 to +0.064 $\mathrm{J\,g^{-1}\,K^{-1}}$, with MD errors of 0.022–0.030 J/g/K from squared-fluctuation correlations. Shared trajectories and harmonic corrections make these cross-checks correlated. Quantitative predictions require stable Hessians, stationary independent trajectories, multiple quenched structures, and validated member propagation.

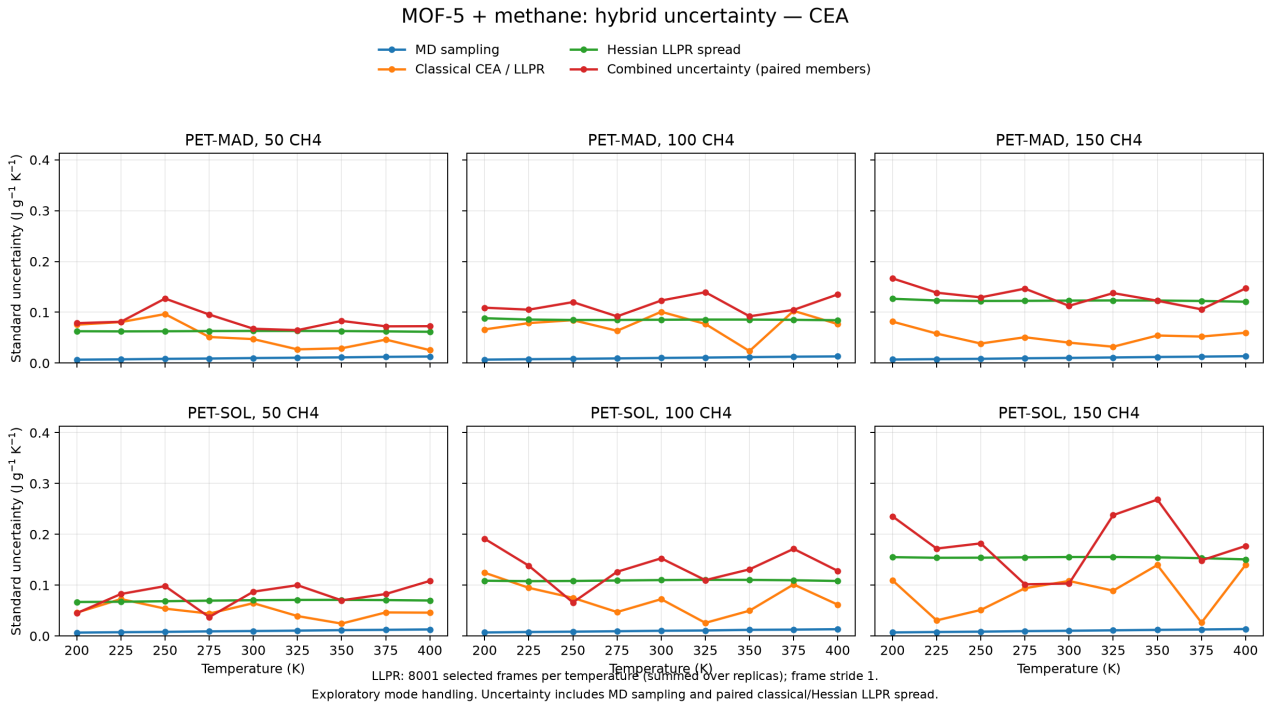


Figure 14: Supplied uncertainty decomposition using refreshed CEA member curves. Every panel uses all 8001 saved production frames at each temperature and 64 LLPR members. The harmonic curve is model spread at one fixed geometry, not an uncertainty across minima. The total preserves classical–harmonic covariance and excludes an unestimated harmonic sampling term. All plotted quantities are standard errors or model standard deviations in J/g/K, not variance fractions.

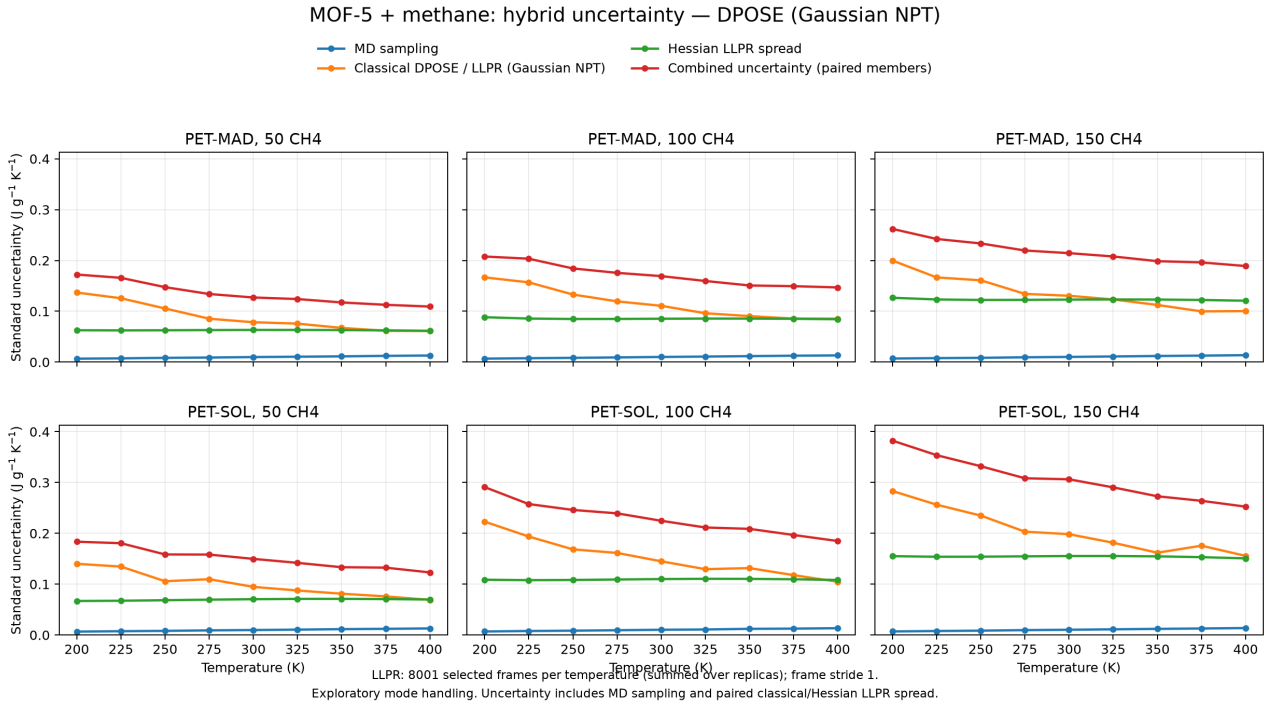


Figure 15: Supplied Gaussian NPT/DPOSE uncertainty decomposition. Central hybrid curves, MD sampling errors, harmonic member corrections, and frame coverage are identical to the CEA comparison; only the classical member spread and its covariance with the harmonic correction change. The smoother orange curves result from a different estimator and do not establish more accurate uncertainty calibration.

5 Recommended production and analysis protocol

Table 7: Production requirements for the workflow in Figure 7. Reference agreement supplements internal convergence; it is not a fitting target.

Calculation	Acceptance and convergence requirements
Empty reference	Relax the equilibrated host directly at fixed cell with the selected checkpoint. Report forces and signed modes; retain a separately normalized harmonic curve.
Loaded NPT	Independent placement/velocity seeds. Check stationary enthalpy, cell, bonds, pores, and guest motion; converge duration, replicas, cell size, timestep, coupling times, and temperature spacing.
Loaded minima	Quench representative independent configurations at fixed cell. Reject unconverged or unstable structures.
Hessians	Converge hops, precision, HVP batching, acoustic treatment, frequency threshold, and supercell/phonon sampling on the correction.
Hybrid assembly	Average accepted loaded corrections; differentiate replica enthalpies; pair LLPR indices. Report MD, minimum sampling, and model uncertainties separately and jointly.

Validate timestep through short NVE energy-drift tests before reservoir coupling, which can mask integration errors. Cell relaxation requires a stress-capable, validated model. One-cell Hessians and selected minima need finite-size and configurational convergence; their unweighted spread is not automatically a free-energy-weighted average. Transfer of the reference hybrid approximation to PET, especially below 200 K, needs validation.

6 Implementation, outputs, and reproducibility

`mof_heat_capacity.protocols.loaded` prepares campaigns; `scripts/md/submit_loaded_md.sh` submits loaded NPT and rejects zero-methane MD. Property stages use the scripts under `scripts/properties/`: `submit_analysis.sh` for diagnostics, `submit_heat_capacity.sh` for fixed-cell relaxation and PET-JAX Hessians, and `submit_hybrid_analysis.sh` for final assembly. Hessian LLPR members run as eight GPU array tasks of eight members each; `analysis.llpr_merge` supplies the final hybrid dependency.

Archives retain structures, signed frequencies, model/run identities, grids, precision, hops, chunking, and rematerialization. Hybrid NPZ/CSV/JSON outputs store the classical term, harmonic correction, sum, uncertainties, and provenance.

Table 8: Uncertainty components, their calculation locations, and their stored outputs.

Quantity	Where it is calculated	Stored result
LLPR energy standard deviation	metatrain LLPR covariance and calibration	`energy_uncertainty`; copied as `analytical_energy_uncertainty_eV` in the per-run archive
CEA member enthalpies and overlap diagnostics	`analysis/uncertainty.py`, function `committee_enthalpy_estimates`	per-run `model_uncertainty.npz`
Classical LLPR C_P standard deviation	`analysis/results.py`, after member-wise replica averaging and temperature differentiation	`model_uncertainty_heat_capacity.npz` and `.csv`
MD sampling standard error	`analysis/statistics.py`, then replica combination and derivative resampling in `analysis/hybrid.py`	`classical_anharmonic_cp_standard_error`
Harmonic sampling standard error	`analysis/hybrid.py`, across independent optimized minima	`harmonic_quantum_correction_standard_error`
Hessian LLPR standard deviation	`analysis/harmonic.py`, after differentiating persistent LLPR readouts at the central minimum	`harmonic_quantum_correction_model_standard_deviation`
Combined standard uncertainty	paired model variance and sampling quadrature in `analysis/hybrid.py`	`approximate_cp_combined_standard_uncertainty`

The scalar energy uncertainty is diagnostic; property errors require complete persistent member curves. Paired MD/harmonic readouts preserve covariance, but Hessians use central-model minima without member-specific relaxation.

Use one node, one task, and one GPU per MD/Hessian job; parallelize independent states, replicas, minima, or member tasks. Retain Slurm IDs and measure representative runtime/memory before adjusting requests:

```
sacct -j JOB_ID --format=JobID,State,Elapsed,MaxRSS,AllocTRES
```

Use distinct outputs for every loading, temperature, seed, and replica.

Record structure/composition and insertion criteria; checkpoint, export, conversion, and environment identities; LLPR dataset/ensemble hashes, calibration, regularizer, seeds, member count, stride, and overlap diagnostics; ensemble/integration settings and restart history; relaxation forces, signed modes, and Hessian controls; and production intervals, replica counts, correlation/block methods, and convergence comparisons.

7 Limitations and interpretation

Path integral molecular dynamics could test the neglected quantum anharmonic contribution, particularly for water-loaded frameworks. It samples quantum nuclear statistics through an auxiliary ring polymer, but is not used for the results reported here. Such validation would require convergence in bead number and trajectory sampling [4, 16].

Table 9: Limits of the present estimates and the evidence needed to resolve them.

Present limitation	Required evidence
33/66 enthalpy drift flags; one replica per state	Longer independent trajectories and stable block/replica statistics.
Imaginary modes in every loaded reference; four failed relaxations	Stable minima and converged signed spectra, sparsity reach, and precision.
One loaded geometry per curve	Multiple independent quenches and sensitivity to harmonic-reference temperature.
Classical MD plus harmonic C_V correction	Validation of quantum anharmonic and pressure–volume effects, especially below 200 K.
Fixed-loading random insertion	Demonstrated equilibration of guest distributions; separate particle-exchange sampling for adsorption equilibrium.
Shallow LLPR; large CEA diagnostics and method-dependent bands	Reference-data calibration and validated propagation; shared representation bias and member-specific relaxation remain outside the current spread.
PET differs from QuickFF; finite-cell/numerical approximations	Loaded-MOF energy/force/stress/curvature benchmarks and size/grid convergence.

8 Conclusions

PET-MAD and PET-SOL give exploratory room-temperature heat capacities rising from about 0.93 to 1.16 J/g/K with methane loading. Harmonic quantization removes 59–63% of the classical term; guest mobility supports retaining anharmonic MD. Broad agreement with digitized reference trends does not establish quantitative accuracy. Stable spectra, independent trajectory/minimum sampling, and validated paired uncertainty propagation are needed before interpreting these curves as converged predictions.

A Transformers and attention

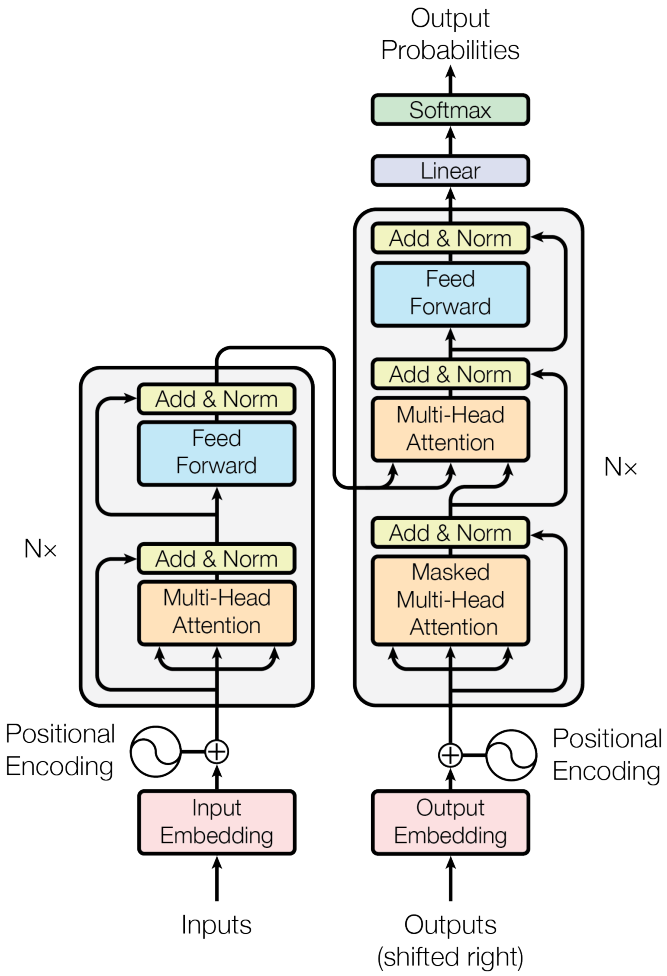


Figure 16: Transformer architecture developed in Ref. [25]. The input matrix $\boldsymbol{X}$ contains the center and neighbor tokens. The query, key, and value matrices $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are computed by learned linear projections. The attention weights $\boldsymbol{A}$ are computed from the scaled dot product of queries and keys, followed by a softmax normalization. The output $\boldsymbol{Y}$ is a weighted sum of the values, which is then passed through a feedforward network and residual connections.

A.1 Tokenization and embedding

The original transformer architecture [22] was designed for sentence translation. But it was then used to other NLP tasks, such as text classification and question answering. Therefore, the transformer architecture takes a text as input. However, a text is a sequence of strings, which is not numerical data and can't be directly processed by a neural network. To convert a sentence into a numerical representation, each word (or punctuation mark) in the sentence is first separated into a token, and then each token is mapped to a vector representation, called an embedding. Each token embedding is a vector living in a high-dimensional space, and the embedding vectors are stacked together to form a matrix $\boldsymbol{X}$. The way to interpret these vectors is that each token is represented as a point in a high-dimensional space, and the distance between two points reflects the semantic similarity between the corresponding tokens. For instance, the tokens "cat" and "dog" are both animals, so their embeddings should be close to each other in the embedding space. In addition, the transformer architecture also uses positional encoding to encode the order of the tokens in the sequence. This is especially important for words that could have different meanings depending on their position in the sentence. For instance, the word "bank" could refer to a financial institution or the side of a river, and its meaning can be inferred from the surrounding words. So the embedding of the word "bank" could have different values depending on its position in the sentence, and the positional encoding helps to capture this information. These embeddings are something that has to be learned from the data, and they are usually initialized randomly and then updated during training. In the case of the PET architecture, the token embeddings are already explained in Section 2.6.1. So what is relevant for us is to understand the next step in the transformer architecture, which is the attention mechanism.

A.2 Attention mechanism

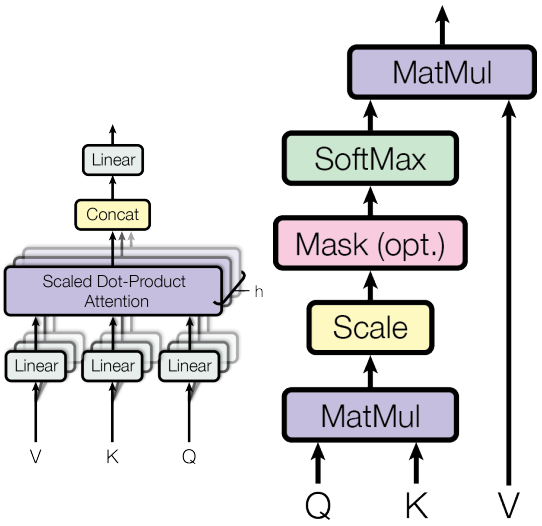


Figure 17: One attention head computes queries, keys, and values from the input token matrix $\boldsymbol{X}$ using learned linear projections. The queries $\boldsymbol{Q}$ are compared with the keys $\boldsymbol{K}$ to produce attention weights $\boldsymbol{A}$, which are then used to compute a weighted sum of the values $\boldsymbol{V}$ to produce the output $\boldsymbol{Y}$.

With our input matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, where n is the number of tokens and d is the embedding dimension, we compute the queries, keys, and values as follows:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V, \tag{107}$$

with $\boldsymbol{W}_Q, \boldsymbol{W}_K \in \mathbb{R}^{d \times d_k}$ and $\boldsymbol{W}_V \in \mathbb{R}^{d \times d_v}$. Thus $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$. These matrices are what the attention mechanism uses to compute the attention matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, which contains the attention weights for each token pair.

$$\boldsymbol{A} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d_k}} + \boldsymbol{M}\right), \tag{108}$$

To better understand what $\boldsymbol{A}$ represents, we can look at its coefficients. Suppose we have a sentence "The animal didn't cross the street because it was too tired". The word "it" refers to "the animal" and not "the street", so the attention weight between "it" and "the animal" should be high, while the attention weight between "it" and "the street" should be low. The term $\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}$ computes the dot product between the queries and keys, which measures the similarity between each token's query and every other token's key. The scaling factor $1/\sqrt{d_k}$ is used to prevent the dot products from growing too large. The $\boldsymbol{M}$ matrix is a mask that can be used to prevent certain tokens from attending to each other. For instance, if we are training the model to predict the next word in a sentence, we don't want the model to know the future words, otherwise it would defeat the purpose of predicting the next word, so we "mask" these words by setting their attention weights to zero. The softmax function is given by

$$\mathrm{softmax}_{\mathrm{row}}(\boldsymbol{Z})_{ab} = \frac{\exp(Z_{ab})}{\sum_{c=1}^{n} \exp(Z_{ac})}, \qquad a, b = 1, \ldots, n, \tag{109}$$

The softmax function normalizes the attention weights so that they sum to one across each row, which means that for each token, the attention weights represent a probability distribution over all other tokens. We add large negative values to the masked positions in $\boldsymbol{M}$ so that their softmax output is effectively zero. After computing $\boldsymbol{A}$, we can compute the output of the attention mechanism as a weighted sum of the values:

$$\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{V}, \tag{110}$$

where $\boldsymbol{Y} \in \mathbb{R}^{n \times d_v}$ is the output of the attention mechanism. Each of the W_Q, W_K, and W_V are weight matrices that are fine-tuned during training. In the PET architecture, masking is not needed because there is no sequential ordering of the tokens, and all tokens can attend to each other.

A.3 Multi-head attention

We can extend the attention mechanism by dividing the input tokens $\boldsymbol{X}$ into subsets $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_H$, such that $d_v = d/H$, and constructing attention matrices $\boldsymbol{A}_1, \ldots, \boldsymbol{A}_H$ for each subset, this is called multi-head attention. We can then concatenate the outputs of each head to form the final output:

$$\mathrm{MHA}(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{Y}_1, \ldots, \boldsymbol{Y}_H)\boldsymbol{W}_O, \qquad \boldsymbol{W}_O \in \mathbb{R}^{Hd_v \times d}. \tag{111}$$

The two main advantages of using multi-head attention instead of simple attention are that it allows to make the process more parallelizable, and it allows the model to learn more complex relationships between the tokens, since it also adds more parameters to the model. A common choice is $d_k = d_v = d/H$ for $d = 256$ and eight heads this gives 32 components per head. These dimensions describe attention projections, rather than the width of a later energy readout head.

A.4 Feedforward operations, residual paths, and normalization

Once the attention is computed, the output $\boldsymbol{Y}$ is passed through a feedforward network (FFN) to produce the final output of the transformer block. Attention exchanges information across token rows. A feedforward network then applies the same nonlinear map to each row independently. With row vectors and an intermediate width d_{ff}, it can be written as

$$\mathrm{FFN}(\boldsymbol{X}) = \sigma\big(\boldsymbol{X}\boldsymbol{W}_1 + \mathbf{1}\boldsymbol{b}_1^{\mathrm{T}}\big)\,\boldsymbol{W}_2 + \mathbf{1}\boldsymbol{b}_2^{\mathrm{T}}, \tag{112}$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{d\times d_{\mathrm{ff}}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}}\times d}$, $\boldsymbol{b}_1 \in \mathbb{R}^{d_{\mathrm{ff}}}$, and $\boldsymbol{b}_2 \in \mathbb{R}^d$. The vector $\mathbf{1} \in \mathbb{R}^n$ broadcasts the same biases over rows, and σ acts componentwise. The expanded intermediate features and nonlinearity let the layer transform the information collected by attention.

Residual connections add a sublayer's output to its input, preserving a direct path for features and gradients. Their addition requires matching feature widths, explaining the final projection in Eq. (111). Layer normalization controls feature scales within each token. For a row $\boldsymbol{x}_a$ with components x_{ap}, a standard form is

$$\begin{aligned} \mu_a &= \frac{1}{d}\sum_{p=1}^{d} x_{ap}, \qquad & v_a &= \frac{1}{d}\sum_{p=1}^{d}(x_{ap}-\mu_a)^2, \\ \mathrm{LN}(\boldsymbol{x}_a)_p &= \gamma_p \frac{x_{ap}-\mu_a}{\sqrt{v_a+\epsilon}} + \beta_p, \qquad & \epsilon &> 0. \end{aligned} \tag{113}$$

The learned scales γ_p and shifts β_p are shared across token rows. The mean and variance are over feature components of one token, rather than over its neighbors. Here v_a is a scalar variance, distinct from the attention value vector $\boldsymbol{v}_a$.

For example, the original transformer's post-normalization arrangement is

$$\begin{aligned} \boldsymbol{X}' &= \mathrm{LN}(\boldsymbol{X} + \mathrm{MHA}(\boldsymbol{X}))\,, \\ \boldsymbol{X}'' &= \mathrm{LN}(\boldsymbol{X}' + \mathrm{FFN}(\boldsymbol{X}'))\,. \end{aligned} \tag{114}$$

A pre-normalization variant instead normalizes the input to each sublayer before adding its output to the residual stream. These are different architectural choices. Equation (114) illustrates the components, not a guarantee that every PET checkpoint uses that normalization order. The PET paper refers to a classical transformer; checkpoint implementations determine the exact order and projection details.

B Sparse automatic differentiation

This appendix explains how to recover a large derivative matrix with fewer automatic differentiation (AD) evaluations. The presentation takes inspiration from the progression in Hill, Dalle, and Montoison's illustrated guide [33]: understand derivative products, identify possible nonzero entries, group columns, and recover their values. We develop an explicit small example before treating symmetric Hessians and the PET/SADMOF workflow. General AD and backpropagation are introduced in Section 2.7.2 and Appendix C.

B.1 The Jacobian, JVPs, and VJPs

For a function $\boldsymbol{f} : \mathbb{R}^m \to \mathbb{R}^n$, the Jacobian matrix is defined as

$$\boldsymbol{J}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_m} & \cdots & \frac{\partial f_n}{\partial x_m} \end{pmatrix} \in \mathbb{R}^{n \times m} \tag{115}$$

We can compute the directional derivative of $\boldsymbol{f}$ at $\boldsymbol{x}$ along a vector $\boldsymbol{v} \in \mathbb{R}^m$ as

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon} \boldsymbol{f}(\boldsymbol{x} + \epsilon \boldsymbol{v}) \bigg|_{\epsilon=0} = \boldsymbol{J}(\boldsymbol{x}) \boldsymbol{v}. \tag{116}$$

This is what we call a *Jacobian–vector product* (JVP). It is the instantaneous rate of change of $\boldsymbol{f}$ along the direction $\boldsymbol{v}$. Since the Jacobian at a point $\boldsymbol{x}$ is a linear map, the directional derivative is linear in $\boldsymbol{v}$. This means that we can recover all the columns of the Jacobian by evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$: $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_m$.

$$\boldsymbol{J}(\boldsymbol{x})_{:,i} = \boldsymbol{J}(\boldsymbol{x}) \boldsymbol{e}_i \tag{117}$$

Let's illustrate this with a simple example. Consider the function $\boldsymbol{f} : \mathbb{R}^4 \to \mathbb{R}^3$

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{pmatrix} x_1^2 + x_2 \\ \sin(x_2) + x_3^2 \\ e^{x_4} \end{pmatrix}, \tag{118}$$

The Jacobian is

$$\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{119}$$

If we try evaluating the JVPs along the standard basis vectors, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_1 = \begin{pmatrix} 2x_1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_2 = \begin{pmatrix} 1 \\ \cos(x_2) \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_3 = \begin{pmatrix} 0 \\ 2x_3 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_4 = \begin{pmatrix} 0 \\ 0 \\ e^{x_4} \end{pmatrix}. \tag{120}$$

Which exactly recovers the columns of the Jacobian in Eq. (119). However, the same result could have been obtained if instead of evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$, we had used the standard basis vectors of $\mathbb{R}^n$ and evaluated the vector-Jacobian products (VJP) along the standard basis vectors of $\mathbb{R}^n$.

$$\boldsymbol{e}_1^{\mathrm{T}} \boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \end{pmatrix}, \quad \boldsymbol{e}_2^{\mathrm{T}} \boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & \cos(x_2) & 2x_3 & 0 \end{pmatrix}, \quad \boldsymbol{e}_3^{\mathrm{T}} \boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{121}$$

This time, we recover the rows of the Jacobian instead of the columns. The advantage of using VJPs here instead of JVPs is that if $n \ll m$, which means we can recover the Jacobian with fewer evaluations. This is particularly useful when dealing with neural networks, where the number of outputs is usually much smaller than the number of inputs. In particular, in the case MLIPs, the number of outputs is only one, the energy of the system, while the number of inputs depends on the number of atoms in the system and can be much larger. For a composition of functions $\boldsymbol{f} = \boldsymbol{h} \circ \boldsymbol{g}$, the chain rule gives the Jacobian as

$$\boldsymbol{J}_f(\boldsymbol{x}) \boldsymbol{v} = \boldsymbol{J}_h(\boldsymbol{g}(\boldsymbol{x})) \big[\boldsymbol{J}_g(\boldsymbol{x}) \boldsymbol{v} \big]. \tag{122}$$

B.2 Recovering multiple columns with a single evaluation

In some cases, we can recover multiple columns of the Jacobian with a single VJP evaluation. For instance, if we evaluate the VJP along the vector $\boldsymbol{v} = \boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{v} = \begin{pmatrix} 2x_1 \\ 2x_3 \\ e^{x_4} \end{pmatrix}. \tag{123}$$

The elements of this vector are the only nonzero entries in the first, third, and fourth columns of the Jacobian. By exploiting the sparsity of the Jacobian, we can recover multiple columns with a single evaluation. This is possible because column 1 has zeros in rows 2 and 3, column 3 has zeros in rows 1 and 3, and column 4 has zeros in rows 1 and 2. In other words, the nonzero entries of these columns do not overlap in any row. The nonoverlapping property is what allows us to recover multiple columns with a single evaluation. The last remaining column can be recovered with a second evaluation along $\boldsymbol{e}_2$. Thus, we can recover the entire Jacobian with only two evaluations instead of four as previously done.

B.3 Sparsity patterns

To exploit the sparsity of the Jacobian, we need to know which entries can be nonzero. One way to represent this information is with a *sparsity pattern* matrix $\boldsymbol{S} \in \{0,1\}^{n\times m}$. Each element in this matrix is equal to 1 if the corresponding entry in the Jacobian can be nonzero, and 0 otherwise. For instance, using the Jacobian in Eq. (119), we can define the following sparsity pattern

$$\boldsymbol{S} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S_{pq} = 0 \implies J_{pq} = 0. \tag{124}$$

The zero coefficients in $\boldsymbol{J}$ are *structural zeros*, which are guaranteed to be zero for all $\boldsymbol{x}$. In $\boldsymbol{S}$, they are also marked as zeros. The other entries in $\boldsymbol{J}$ are *possible nonzeros*, which can be nonzero for some $\boldsymbol{x}$, but aren't guaranteed to be nonzero for all $\boldsymbol{x}$. In $\boldsymbol{S}$, they are marked as ones. For instance, $J_{11} = 2x_1$ happens to vanish at $x_1 = 0$, but it still receives a 1 in this pattern because it is nonzero for other values of x_1.

B.4 Graph coloring

To better understand which columns can be grouped together, we can represent the sparsity pattern as a graph. Each column in $\boldsymbol{S}$ corresponds to a vertex in the graph and an edge connects two vertices if they have one or more nonzero entries in the same row of $\boldsymbol{S}$. In our example, columns 1 and 2 have a nonzero entry in row 1, so they are connected by an edge. Same for columns 2 and 3, which have a nonzero entry in row 2. The set of vertices and edges forms a graph $G = (V, E)$, where $V = \{1, 2, 3, 4\}$ is the set of vertices (columns of $\boldsymbol{S}$) and $E = \{\{1, 2\}, \{2, 3\}\}$ is the set of edges (connections between columns). We can represent a graph with a diagram, where vertices are drawn as circles and edges as lines connecting them.

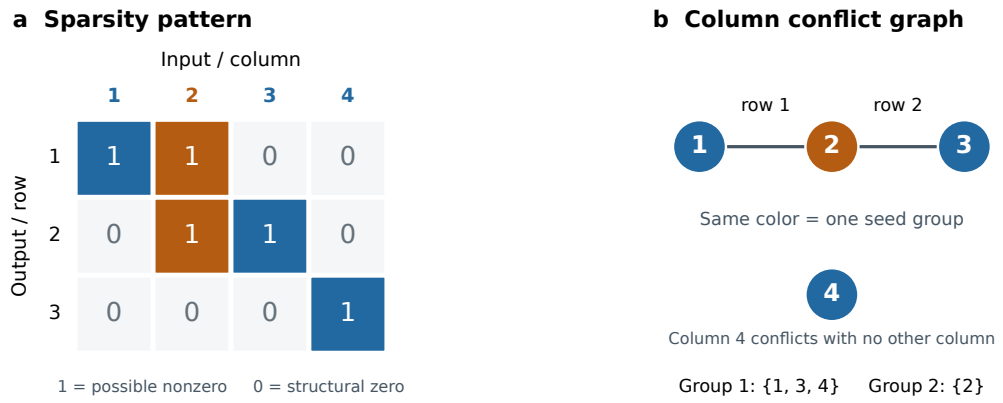


Figure 18: The same four-input example viewed as a pattern and as a graph. Filled cells mark possible nonzero entries. Graph edges record conflicts between columns, with the responsible row written beside each edge. Blue columns 1, 3, and 4 have no mutual conflicts and share one seed; orange column 2 uses another. A color denotes a group of columns.

The next step is to assign a color to each vertex in the graph such that no two adjacent vertices share the same color. This process is called *graph coloring*. The goal is to find a coloring that uses the smallest number of colors

possible $\chi(G)$, which is called the *chromatic number* of the graph., The number of colors in the graph corresponds to the number of JVP evaluations needed to recover all columns of $\boldsymbol{J}$. The columns of the same color have no conflicts and can be computed together in a single JVP evaluation. In our example, $\chi(G) = 2$, as shown in Fig. 18. Columns 1, 3, and 4 can be assigned the same color (blue), while column 2 can be assigned another color (orange). As we saw previously, we could recover columns 1, 3, and 4 with a single JVP evaluation along $\boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, and column 2 with another JVP evaluation along $\boldsymbol{e}_2$. Which is consistent with the coloring we found. For this simple example, the chromatic number is easy to find by inspection. But in general, finding the chromatic number of a graph is an NP-hard problem. Therefore, we need to use certain algorithms to find a coloring that uses a small number of colors, even if it is not the minimum possible.

B.5 Decompression

After finding a coloring, we can evaluate the JVPs along the vectors belonging to each color and concatenate the results into a compressed matrix $\boldsymbol{Y}$. In our example, we have the following

$$\boldsymbol{v}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{v}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \qquad \boldsymbol{Y}(\boldsymbol{x}) = \begin{bmatrix} \boldsymbol{J}(\boldsymbol{x})\boldsymbol{v}_1 & \boldsymbol{J}(\boldsymbol{x})\boldsymbol{v}_2 \end{bmatrix} = \begin{pmatrix} 2x_1 & 1 \\ 2x_3 & cos(x_2) \\ e^{x_4} & 0 \end{pmatrix}. \tag{125}$$

This compressed matrix contains all the information needed to recover the original Jacobian. Instead of storing a full 3×4 Jacobian, we only need to store a 3×2 compressed matrix. To recover the original $\boldsymbol{J}$, we can use the coloring information to unpack $\boldsymbol{Y}$. In the first column of $\boldsymbol{Y}$, we have $J_{11} = 2x_1$, $J_{23} = 2x_3$, and $J_{34} = e^{x_4}$, and in the second column of $\boldsymbol{Y}$, we have $J_{12} = 1$, $J_{22} = cos(x_2)$ and $J_{32} = 0$. The other entries in $\boldsymbol{J}$ are structural zeros. Recovering $\boldsymbol{J}$ from $\boldsymbol{Y}$ is called *decompression*.

B.6 The Hessian and HVPs

For a function $f : \mathbb{R}^m \to \mathbb{R}$, the Hessian matrix is defined as

$$\boldsymbol{H}(\boldsymbol{x}) = \frac{\partial^2 f}{\partial \boldsymbol{x}^2}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_m \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_m^2} \end{pmatrix} \in \mathbb{R}^{m \times m}. \tag{126}$$

The methods we have discussed for the Jacobian can also be applied to the Hessian. In fact, it can be applied to any sparse matrix, regardless of whether it is a Jacobian or Hessian. Therefore, the same coloring and decompression techniques can be used to recover the Hessian with fewer evaluations.

Why the physical neighbor graph is insufficient. Consider a symmetric matrix with entries only on its diagonal and the two adjacent diagonals, called a *tridiagonal* matrix:

$$\boldsymbol{M} = \begin{pmatrix} a & b & 0 & 0 \\ b & c & d & 0 \\ 0 & d & e & f \\ 0 & 0 & f & g \end{pmatrix}. \tag{127}$$

Its off-diagonal pattern forms a chain $1-2-3-4$: an edge means that the corresponding off-diagonal matrix entry may be nonzero. Coloring this chain alternately with $(1, 2, 1, 2)$ separates immediate neighbors. But columns 1 and 3 both contribute to row 2, so adding them mixes b and d. The chain is the matrix's *sparsity graph*; direct column recovery needs the *conflict graph*, which must also connect 1 to 3 and 2 to 4. Those pairs are two edges apart in the chain but overlap in a matrix row.

A valid coloring of the conflict graph is $(1, 2, 3, 1)$. Its seeds and products are

$$\boldsymbol{V} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \qquad \boldsymbol{MV} = \begin{pmatrix} a & b & 0 \\ b & c & d \\ f & d & e \\ g & 0 & f \end{pmatrix}. \tag{128}$$

Only columns 1 and 4 share a seed. Their entries occupy rows 1–2 and 3–4, respectively, so they can be unpacked without ambiguity. Three products recover the matrix rather than four. On a longer tridiagonal chain, repeating three colors still separates all column conflicts.

B.7 Star coloring: using symmetry to recover an entry from either side

A Hessian is symmetric when the scalar energy has continuous second derivatives: $H_{pq} = H_{qp}$. This duplicates each off-diagonal value across the diagonal. Suppose one compressed sum mixes H_{pq} with other entries. If its counterpart H_{qp} can be read separately from another row, we can recover both positions from that one value. Direct symmetric recovery therefore needs only *one unambiguous orientation* per off-diagonal entry [18].

The additional coloring rule. Star coloring acts on the symmetric matrix's off-diagonal sparsity graph, not on the larger column-intersection graph. It imposes two conditions:

1. Adjacent vertices have different colors, as in a proper coloring.
2. Every path through four distinct vertices uses at least three colors.

A *path* is a sequence of vertices connected by successive edges. Four vertices make three edges. Thus a blue–orange–blue–orange path is forbidden, whereas blue–orange–blue–green is allowed. These rules ensure that at least one orientation of every off-diagonal entry is unambiguous [19].

The name comes from looking at the edges between any two chosen colors. Every connected piece must form a *star*: one central vertex, or hub, connected to outer vertices, or spokes, with no edges between the spokes. An isolated edge or vertex also satisfies the condition. A spoke has only one neighbor of the hub's color. Its compressed row therefore isolates the spoke–hub entry, even if the hub's row sums several spokes.

Recovering the chain example. The alternating coloring $(1, 2, 1, 2)$ fails the four-vertex rule. For its middle entry $d = M_{23} = M_{32}$, the two candidate compressed values are $M_{21} + M_{23} = b + d$ and $M_{32} + M_{34} = d + f$. Neither directly isolates d. A star coloring avoids the need to disentangle such sums.

Change the last color to obtain $(1, 2, 1, 3)$. The products now become

$$\boldsymbol{W} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \boldsymbol{B} = \boldsymbol{M}\boldsymbol{W} = \begin{pmatrix} a & b & 0 \\ b+d & c & 0 \\ e & d & f \\ f & 0 & g \end{pmatrix}. \tag{129}$$

Columns 1 and 3 still share a seed, so $B_{21} = b + d$ is still mixed. We simply do not use that entry for recovery. Instead, $b = B_{12}$ and $d = B_{32}$ give the two values separately; symmetry fills their transposed positions. The other entries follow as $a = B_{11}$, $c = B_{22}$, $e = B_{31}$, $f = B_{33} = B_{41}$, and $g = B_{43}$. This example uses three colors with either recovery strategy. On other patterns, symmetry can reduce the number of required colors.

Why the rule works. For an allowed pair p, q, inspect all neighbors of p with the same color as q. If q is the only one, $Y_{p,c(q)}$ contains H_{pq} alone. Otherwise, inspect the reverse orientation. If both orientations were mixed, there would be another neighbor r of p colored like q, and another neighbor s of q colored like p. The path $r - p - q - s$ would then alternate between two colors, violating the star rule. Proper coloring ensures that these are four distinct vertices. For a diagonal entry, no adjacent vertex shares its color, so $H_{pp} = Y_{p,c(p)}$ directly.

A greedy star-coloring algorithm must check both conditions before accepting a color. Checking only the colors of immediate neighbors would produce an ordinary proper coloring, which is insufficient for this recovery scheme. Sparsity, the coloring rule, and the decompression rule must therefore be chosen consistently.

B.8 From PET neighborhoods to a Hessian pattern

SADMOF constructs an atom-pair pattern from PET's selected-neighbor graph, then expands each pair into a Cartesian 3×3 block [34]. The reasoning starts from the energy decomposition $U = \sum_t \varepsilon_t$:

$$H_{i\alpha,j\beta} = \sum_t \frac{\partial^2 \varepsilon_t}{\partial R_{i\alpha} \partial R_{j\beta}}. \tag{130}$$

A contribution is possible only if both atoms influence the same energy term. Two atoms therefore need not be direct neighbors to have a nonzero Hessian block. For a node readout with L rounds of message passing, both can lie up to L hops from the center, giving a separation of at most $K = 2L$ hops. For the PET edge readout analyzed in Ref. [34], the term spans the L-hop neighborhoods of two adjacent endpoints; joining the two paths adds one edge and gives $K = 2L + 1$. These are architectural reach bounds under the paper's derivative convention, not statements that every permitted entry is numerically nonzero.

The *adjacency matrix* $\boldsymbol{A}$ records the selected-neighbor graph: $A_{ij} = 1$ if atoms i and j are connected, and 0 otherwise. Use an undirected graph, so each connection can be followed in either direction. A *hop* means following one edge. To include all atom pairs separated by at most k hops, build

$$\boldsymbol{P}^{(k)} = \boldsymbol{I} \vee \boldsymbol{A} \vee \boldsymbol{A}^2 \vee \cdots \vee \boldsymbol{A}^k, \tag{131}$$

The powers here use *Boolean arithmetic*: multiplication means AND, and addition means OR. Thus $(\boldsymbol{A}^2)_{ij} = 1$ if there is at least one intermediate atom connecting i to j in two steps. We care whether a path exists, rather than how many such paths exist. The symbol $\vee$ combines the possibilities from different path lengths. Thus $P_{ij}^{(k)} = 1$ when atoms i, j are connected by at most k edges, including $i = j$ through the identity matrix $\boldsymbol{I}$, whose diagonal entries are 1 and other entries are 0. A hop is a graph step, not a fixed Cartesian distance. With $p = 3(i-1)+\alpha$ and $q = 3(j-1)+\beta$, where $\alpha, \beta \in \{1, 2, 3\}$, the Cartesian pattern is

$$S_{pq} = P_{ij}^{(k)}, \qquad \boldsymbol{S} = \boldsymbol{P}^{(k)} \otimes \mathbf{1}_{3\times 3}. \tag{132}$$

Here $\mathbf{1}_{3\times 3}$ is the all-ones matrix, and $\otimes$ denotes the Kronecker product: replace every 1 in the atom-pair pattern by a 3×3 block of ones and every 0 by a block of zeros. Each block allows all nine combinations of the two atoms' x, y, and z coordinates. At $k = K$ the pattern contains all couplings under the chosen derivative convention; $k < K$ truncates it. Additional nonlocal terms need separate analysis. Structural sparsity requires

$$S_{pq} = 0 \quad \Longrightarrow \quad H_{pq} = 0. \tag{133}$$

An overestimate costs entries and colors. An underestimate can both discard curvature and contaminate retained entries. Mixed second derivatives depend on nonlinear readout, not just direct interaction edges.

An optional reduction in coloring work. Appendix D of Ref. [34] uses this block structure to color the N-vertex atom-pair pattern first. For one-based atom colors $c_i \in \{1, \ldots, n_{\text{atom}}\}$, assign Cartesian colors

$$c\big(3(i-1)+\alpha\big) = 3(c_i - 1) + \alpha, \qquad \alpha \in \{1, 2, 3\}. \tag{134}$$

A valid star coloring of the atom-pair pattern then gives a valid star coloring of the Cartesian pattern, with $3n_{\text{atom}}$ coordinate colors. For example, atom color 1 becomes coordinate colors 1, 2, and 3, while atom color 2 becomes colors 4, 5, and 6. The three directions of an atom must receive distinct colors because its on-site block is generally dense. This reduces the graph size used in preparation. The current project delegates coloring to ASDEX; this block optimization is described in the paper and is not a separate atom-coloring step implemented by the project wrapper.

B.9 The complete SADMOF procedure in this project

The implementation in `mof_heat_capacity.analysis.harmonic` separates preparation from evaluation as follows.

1. **Prepare the reference and energy function.** Use a fixed-cell optimized structure, fixed species, and fixed model parameters. Convert the structure to PET-JAX positions, cell, and graph arrays, retaining any padding required by the implementation (extra placeholder entries that keep array sizes consistent). The energy function uses `no_shadow=True`: derivatives through the adaptive neighbor-selection cutoff are stopped, and the selected graph defines the locality assumed by the pattern. Ordinary position-dependent PET operations still differentiate according to that implemented convention. The sparse path rejects `shadow=true`; comparisons to MD forces or a dense Hessian must use matching derivative conventions.

2. **Build the assumed pattern.** Take the selected center–neighbor pairs and call SADMOF's `sparsity_pattern` with the requested `hops`. This implements the locality-based preparation described above; the code does not estimate structural zeros by thresholding a previously evaluated Hessian.

3. **Prepare colors and recovery indices.** Pass the pattern to ASDEX's `hessian_coloring_from_sparsity` with `mode="fwd_over_rev"`. Star coloring determines the seed groups and which compressed orientation supplies each Hessian entry. The recovery indices are a stored lookup table telling decompression which row and color column to read for each allowed Hessian entry. They depend on the pattern, not the numerical derivative values at the current point.

4. **Compile and evaluate the directional derivatives.** Construct SADMOF's Hessian function and compile it with JAX. For each color a, evaluate $\boldsymbol{y}_a = \boldsymbol{H}\boldsymbol{v}_a$ by forward AD through the reverse-mode gradient. `chunk_size` controls how many directions are batched; `remat` trades recomputation of intermediates for lower memory use. Neither changes the mathematical sparsity pattern.

5. **Decompress the Hessian.** Use the stored pattern and recovery indices to extract entries from the compressed products and return a sparse matrix. For a conservative pattern this recovers the implemented Hessian up to floating-point error; a truncated pattern introduces the errors discussed below.

6. **Prepare normal modes.** Convert the result to a dense array, remove padded atoms, and reshape to $3N \times 3N$. The subsequent analysis symmetrizes the matrix, applies the configured acoustic-sum-rule treatment, mass-weights it, and diagonalizes it as described in Section 2.7.1. These postprocessing steps are distinct from sparse AD recovery.

For LLPR members evaluated at the same geometry and with the same architecture and neighbor selection, the pattern, coloring, and compiled function can be reused while changing the parameter values. New numerical products are still needed for every member. Changing the geometry or selection can introduce new couplings; preparation must then be rebuilt or its pattern shown to remain a conservative superset.

Table 10: Numerical controls for Hessian evaluation. Template settings are not established convergence values; archived results use float64.

Control	Effect	Template
`hops`	Changes the assumed sparsity approximation	3
`chunk_size`	Batches colored directions; affects memory/throughput	1
`remat`	Recomputes intermediates to reduce memory	true
`dtype`	Sets floating-point precision	float32

B.10 Accuracy, truncation, and computational cost

The pattern must include every actual coupling for exact recovery. For example, if the matrix in Eq. (127) also had $M_{24} = M_{42} = \delta$ but the pattern omitted this pair, the direct coloring would still combine columns 1 and 4 and incorrectly recover M_{21} as $b + \delta$. Thus an underestimated pattern can corrupt retained entries as well as omit missing ones.

For the three-block PET models used here, the paper's bound is $K = 7$. The configured `hops=3` therefore defines a truncated approximation, not an established exact pattern. The energy function used for each HVP still contains the full message-passing computation: reducing the assumed pattern does not remove the omitted couplings from those products. Consequently, truncated reconstruction is not equivalent to calculating a dense Hessian and simply setting distant blocks to zero.

Let $\widehat{\boldsymbol{H}}_k$ be the recovered Hessian before sum-rule postprocessing and let $\boldsymbol{S}_k$ be its Cartesian binary mask. Writing $\odot$ for entrywise multiplication separates the error into

$$\begin{aligned} \Delta\boldsymbol{H}_{\mathrm{disc}} &= -(\mathbf{1} - \boldsymbol{S}_k) \odot \boldsymbol{H}, \\ \Delta\boldsymbol{H}_{\mathrm{cont}} &= \boldsymbol{S}_k \odot (\widehat{\boldsymbol{H}}_k - \boldsymbol{H}), \\ \|\widehat{\boldsymbol{H}}_k - \boldsymbol{H}\|_F^2 &= \|\Delta\boldsymbol{H}_{\mathrm{disc}}\|_F^2 + \|\Delta\boldsymbol{H}_{\mathrm{cont}}\|_F^2. \end{aligned} \tag{135}$$

Here $\mathbf{1}$ is the all-ones matrix of the same shape, and $\|\boldsymbol{A}\|_F^2 = \sum_{pq} A_{pq}^2$ is the squared Frobenius norm, a measure of the total squared size of all matrix entries. The first term contains discarded entries, and the second contains contamination of retained entries. They occupy different matrix positions, so their squared entries add without cross terms, giving the norm identity [34]. Neither symmetrization nor enforcing the acoustic sum rule generally removes these errors.

Validation should compare with a feasible dense Hessian or an exact-pattern result under the same derivative convention, and increase `hops` while monitoring signed frequencies and the heat-capacity correction across the temperature range. Precision must be checked separately, especially for low-frequency modes. Convergence can be nonmonotonic near mode thresholds; periodic image folding also requires cell/supercell convergence. Keep separately tagged outputs for these comparisons. Rapid decay observed in the preprint's benchmarks does not establish convergence for loaded MOF-5.

If one energy evaluation costs $\mathcal{O}(N)$ at fixed neighbor count and architecture, the differentiated work scales roughly as $\mathcal{O}(Nn_c)$ instead of $\mathcal{O}(N^2)$ for dense coordinate seeding. This estimate assumes a bounded cost factor for each HVP and excludes preparation, compilation, and reconstruction. Fewer colors need not translate into the same factor of wall-time saving: batching, device utilization, and memory matter. Finally, this project densifies the Hessian and computes the full spectrum, retaining quadratic matrix storage and cubic dense eigendecomposition cost. Sparse AD accelerates the derivative stage, not every later part of the calculation.

C Computational graphs, backpropagation, and parameter updates

This appendix expands the training overview in Section 2.5.5. It follows the calculation in order: construct and evaluate the computational graph, propagate sensitivities backward, collect parameter gradients, and apply an optimizer update. The scalar example and the multilayer equations are illustrations of the same chain-rule procedure. The connection to PET then explains how shared parameters, forces, and second derivatives enter.

C.1 Step 1: construct and evaluate the computational graph

Backpropagation computes the loss gradients required by an optimizer such as gradient descent. It applies the chain rule backward through the operations recorded during the forward pass, reusing intermediate values. It therefore complements gradient descent: backpropagation calculates the gradients, and the optimizer uses them to update the parameters.

Representing dependencies. In practice, backpropagation is implemented using reverse-mode automatic differentiation (AD). A computational graph decomposes the network and loss into elementary operations, such as multiplication, addition, and activation. Its nodes represent input values or the results of operations, and directed edges record which values an operation depends on. The graph for a fixed forward evaluation is directed and acyclic: a value must be available before an operation that uses it can be evaluated. Repeating a network layer creates further operations in this graph rather than a circular dependency.

Input data and trainable parameters are both inputs to the computation; these input nodes are called leaves. During training, the data and reference targets are held fixed while the optimizer adjusts the parameters. The forward pass evaluates the graph in dependency order to obtain the prediction and loss. It also retains the intermediate values required for differentiation; some can instead be recomputed during the backward pass to reduce memory use. For example, the local derivatives of $u = wx$ are $\partial u/\partial w = x$ and $\partial u/\partial x = w$, so the backward calculation needs the forward values of x and w.

Each elementary operation has a derivative rule supplied by the AD system. The backward pass applies these rules to the recorded dependencies, automatically composing them through the chain rule. The complete derivative therefore need not be derived or programmed separately for each architecture. A computational graph records how the calculation is performed; it is different from the atomic neighbor graph used to describe a material.

C.2 Step 2: propagate sensitivities in reverse order

For scalar intermediate values v_i, define the adjoint $\bar{v}_i = \partial\mathcal{L}/\partial v_i$. After the forward evaluation, the reverse pass starts with $\bar{\mathcal{L}} = 1$ and all other adjoints initialized to zero. It visits operations in reverse dependency order. For every input v_i of an operation producing v_j, it accumulates

$$\bar{v}_i \mathrel{+}= \bar{v}_j \frac{\partial v_j}{\partial v_i}. \tag{136}$$

The accumulation matters when one value contributes to the loss through several paths: all contributions must be added. For vector operations, the corresponding rule multiplies an incoming adjoint by a transposed local Jacobian, usually without constructing that Jacobian explicitly. The activations retained or recomputed from the forward pass supply the values needed to evaluate these local derivatives.

For a vector-valued operation $\boldsymbol{v}_j = \phi(\boldsymbol{v}_i)$ and column-vector adjoints, this rule is

$$\bar{\boldsymbol{v}}_i \mathrel{+}= \left(\frac{\partial \boldsymbol{v}_j}{\partial \boldsymbol{v}_i}\right)^{\mathrm{T}} \bar{\boldsymbol{v}}_j. \tag{137}$$

The incoming adjoint describes how the final loss responds to the operation's output. The local derivative converts that sensitivity into sensitivities to its inputs. This is a vector–Jacobian product written with column vectors. If $\boldsymbol{v}_i$ has n components and $\boldsymbol{v}_j$ has m, the local Jacobian is $m \times n$ and the resulting adjoint has n components.

For scalar operations, some elementary rules are shown in Table 11. Each contribution is added to the input's current adjoint, because the same input can be used elsewhere.

Reverse dependency order is necessary: before processing an operation, all later operations that consume its output must have contributed to that output's adjoint. A gradient can therefore be wrong even when every local derivative is correct if contributions along different paths are overwritten instead of accumulated.

A single reverse pass computes the gradient of a scalar loss with respect to all model parameters, making reverse-mode AD well suited to training networks with many weights. It applies the chain rule to the implemented operations rather than approximating derivatives by finite differences or constructing a global symbolic expression. The resulting derivatives are subject to floating-point error [32]. The reverse sweep computes derivatives at the current parameter values; it does not change those values. Updating parameters is a subsequent optimizer step.

Table 11: Local reverse-mode rules. The forward values of the inputs or outputs supply the factors used in the reverse pass.

Forward operation	Contribution to $\bar{u}$	Contribution to $\bar{v}$
$z = u + v$	$\bar{z}$	$\bar{z}$
$z = uv$	$\bar{z}\, v$	$\bar{z}\, u$
$z = \tanh u$	$\bar{z}(1 - z^2)$	—
$z = \frac{1}{2}u^2$	$\bar{z}\, u$	—

C.3 A worked example: from forward values to parameter gradients

Consider an illustrative network consisting of one scalar neuron with a tanh output and a half-squared-error loss:

$$\begin{aligned} u &= wx, & z &= u + b, \\ \hat{y} &= \tanh z, & r &= \hat{y} - y, \\ l &= \tfrac{1}{2} r^2. \end{aligned} \tag{138}$$

Here x is the input, w and b are trainable parameters, and y is the reference target. This small example illustrates the differentiation rules; it is not the full network architecture of Figure 4. For $x = 2$, $w = 0.5$, $b = -1$, and $y = 1$, the forward pass gives $u = 1$, $z = 0$, $\hat{y} = 0$, $r = -1$, and $l = 0.5$. Figure 19 shows the computation and its reverse pass.

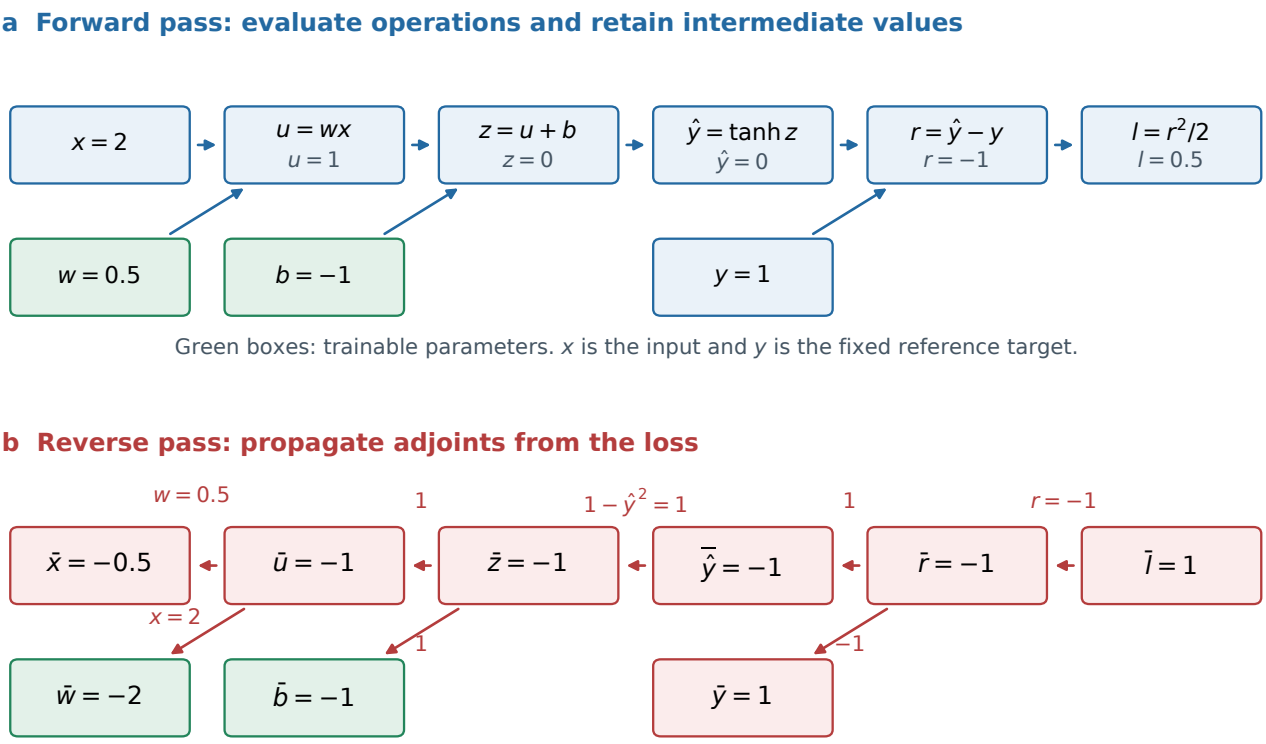


Figure 19: Computational graph for the scalar-neuron example in Eq. (138). (a) The forward pass follows dependencies to evaluate the prediction and loss; boxes show operations and their numerical values. Green boxes identify trainable parameters. (b) The reverse pass starts with $\bar{l} = 1$ and propagates adjoints $\bar{v} = \partial l / \partial v$. Arrow labels give local derivatives at the forward values; multiplying them by incoming adjoints gives the sensitivities in the boxes. Input and target adjoints are shown for completeness, but only w and b are updated during training.

Starting with $\bar{l} = 1$, the reverse rules for this graph give

$$\begin{aligned} \bar{r} &= \bar{l}\, r, & \bar{\hat{y}} &= \bar{r}, \\ \bar{z} &= \bar{\hat{y}}\,(1 - \hat{y}^2), & \bar{u} &= \bar{z}, \\ \bar{w} &= \bar{u}\, x, & \bar{b} &= \bar{z}. \end{aligned} \tag{139}$$

The activation derivative $1 - \hat{y}^2$ uses the output retained from the forward pass. At the chosen values, $\bar{r} = \bar{\hat{y}} = \bar{z} = \bar{u} = -1$, so $\partial l / \partial w = -2$ and $\partial l / \partial b = -1$. The other input derivatives are $\bar{x} = \bar{u}\, w = -0.5$ and $\bar{y} = -\bar{r} = 1$.

With learning rate η, gradient descent would update $w \leftarrow 0.5 + 2\eta$ and $b \leftarrow -1 + \eta$; it leaves x and y fixed. Thus backpropagation supplies the sensitivities used by the optimizer.

If a parameter is reused in several operations, its graph has several outgoing dependency edges. Its final adjoint is the sum of the contributions arriving along all reverse paths, as specified by Eq. (136). This is relevant to shared weights in GNNs and transformers: using one learned map for many atoms or tokens means that all those uses contribute to the same parameter gradient.

C.4 Step 3: collect gradients through network layers

The layer formulas below are the vector form of the same procedure.

For a single scalar-output training example, consider the unregularized loss $l = \frac{1}{2}(\hat{y} - y)^2$. The factor $\frac{1}{2}$ simplifies the derivatives. The derivative at the output is the signed residual, $r = \partial l/\partial \hat{y} = \hat{y} - y$. Consequently, $\partial l/\partial \boldsymbol{w}_{\text{out}} = r\boldsymbol{h}^{(L)}$ and $\partial l/\partial b_{\text{out}} = r$.

To propagate this information into the hidden layers, define $\boldsymbol{\delta}^{(\ell)} = \partial l/\partial \boldsymbol{z}^{(\ell)}$. For the final hidden layer and the preceding layers, the chain rule gives

$$\begin{aligned} \boldsymbol{\delta}^{(L)} &= r\boldsymbol{w}_{\text{out}} \odot \sigma'\left(\boldsymbol{z}^{(L)}\right), \\ \boldsymbol{\delta}^{(\ell)} &= \left[(\boldsymbol{W}^{(\ell)})^{\mathrm{T}}\boldsymbol{\delta}^{(\ell+1)}\right] \odot \sigma'\left(\boldsymbol{z}^{(\ell)}\right), \quad \ell = L-1, \ldots, 1. \end{aligned} \tag{140}$$

The symbol $\odot$ denotes componentwise multiplication. The transposed weight matrix combines the sensitivities from the next layer, and the activation derivative accounts for how each neuron responds to its input. To check the dimensions, let layer ℓ have width d_ℓ. Then $\boldsymbol{W}^{(\ell)} \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$, $\boldsymbol{h}^{(\ell)} \in \mathbb{R}^{d_\ell}$, and $\boldsymbol{\delta}^{(\ell+1)} \in \mathbb{R}^{d_{\ell+1}}$. The parameter gradients then follow directly:

$$\frac{\partial l}{\partial \boldsymbol{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell+1)}(\boldsymbol{h}^{(\ell)})^{\mathrm{T}}, \qquad \frac{\partial l}{\partial \boldsymbol{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell+1)}, \quad \ell = 0, \ldots, L-1. \tag{141}$$

Each weight gradient is the product of the incoming feature and the sensitivity of the receiving neuron. Thus, the backward pass assigns a loss derivative to every trainable connection and bias.

The outer product has the same $d_{\ell+1} \times d_\ell$ shape as the weight matrix. In particular, the entry for a connection from feature j to neuron i is $\partial l/\partial W_{ij}^{(\ell)} = \delta_i^{(\ell+1)} h_j^{(\ell)}$. The bias gradient has the same shape as the receiving layer's bias. This is how the graph's local sensitivities become derivatives for every trainable array.

C.5 Step 4: combine examples and update parameters

For a batch of B examples with the illustrative half-squared losses $l_n = \frac{1}{2}(\hat{y}_n - y_n)^2$, define

$$\mathcal{L}_B = \frac{1}{B}\sum_{n=1}^{B} l_n + \lambda\mathcal{R}(\theta), \qquad \nabla_\theta \mathcal{L}_B = \frac{1}{B}\sum_{n=1}^{B} \nabla_\theta l_n + \lambda\nabla_\theta\mathcal{R}. \tag{142}$$

The averaging factor scales each example's contribution. A batch can be evaluated using batched array operations; the expression does not require running a separate optimizer update for every example. The mean squared error in Eq. (76) omits the factor $\frac{1}{2}$, so its data gradient is twice the averaged half-squared-error gradient above, with the same regularization term.

After the backward pass, gradient descent updates all weights and biases:

$$\begin{aligned} \boldsymbol{W}^{(\ell)} &\leftarrow \boldsymbol{W}^{(\ell)} - \eta\nabla_{\boldsymbol{W}^{(\ell)}}\mathcal{L}, \\ \boldsymbol{b}^{(\ell)} &\leftarrow \boldsymbol{b}^{(\ell)} - \eta\nabla_{\boldsymbol{b}^{(\ell)}}\mathcal{L}. \end{aligned} \tag{143}$$

The forward pass, backward pass, and parameter update are repeated during training. Once training is complete, inference evaluates the network with fixed parameters.

For a single update, the forward and reverse passes use the same parameter values; only after the gradient is available does the optimizer change them. An implementation that accumulates gradients in persistent buffers must clear those buffers before starting a new independent update, unless accumulation across batches is intentional. Adam and related optimizers use the same loss gradients but also maintain update statistics. Their parameter step therefore differs from the simple gradient-descent rule above.

C.6 Shared weights, residual paths, and the PET computational graph

PET applies learned maps repeatedly across atoms, neighbors, and tokens. Each application has its own intermediate activations, while references to a shared parameter accumulate into one gradient. For example, if $u_1 = wx_1$, $u_2 = wx_2$, and a loss depends on both u_1 and u_2, then

$$\frac{\partial \mathcal{L}}{\partial w} = \bar{u}_1 x_1 + \bar{u}_2 x_2. \tag{144}$$

The same principle applies when the shared map is a species embedding, attention projection, feedforward layer, or readout head. Parameter sharing changes where contributions are added, rather than requiring a new differentiation rule.

Residual connections create another kind of branching. For $\boldsymbol{z} = \boldsymbol{h} + F_\theta(\boldsymbol{h})$, the input receives contributions through both the identity path and the learned map:

$$\bar{\boldsymbol{h}} = \bar{\boldsymbol{z}} + \boldsymbol{J}_{F_\theta}(\boldsymbol{h})^{\mathrm{T}}\bar{\boldsymbol{z}}. \tag{145}$$

This expression gives the contribution from this residual block; other uses of $\boldsymbol{h}$ contribute in addition. The residual route provides a direct sensitivity path through the stack, although it alone does not guarantee successful optimization. Attention creates dependencies between tokens, so its backward calculation must also differentiate the query, key, value, normalization, and cutoff operations described in Appendix A.

The reverse pass needs suitable forward values, so memory use grows with the stored intermediates. Recomputing selected forward operations during backpropagation, often called checkpointing or rematerialization, trades additional work for lower memory use. This is the same general tradeoff used by the Hessian evaluation controls in Appendix B.9.

C.7 From loss gradients to forces and second derivatives

The same AD machinery also computes physical derivatives of an MLIP. During training, the independent variables are the model parameters and the differentiated scalar is the loss. For force evaluation, the parameters are held fixed and the atomic coordinates are differentiated instead: $\boldsymbol{F} = -\nabla_{\boldsymbol{R}} U_\theta$. A reverse pass through the energy calculation therefore returns all force components.

Provided the operations support second derivatives, the energy-gradient calculation can itself be differentiated to obtain Hessian information. For example, forward-mode AD applied to the reverse-mode energy gradient computes a Hessian–vector product $\boldsymbol{H}\boldsymbol{v}$. Recovering the full Hessian requires multiple such products; one energy backward pass alone gives only the gradient. Section 2.7.2 develops this construction for the vibrational calculations. Training on forces also differentiates through an energy derivative, in that case producing mixed derivatives with respect to coordinates and parameters.

For example, for one configuration with $M = 3N$ force components, consider the illustrative force-only loss

$$\mathcal{L}_F = \frac{w_F}{2M}\sum_{q=1}^{M}(\hat{F}_q - F_q^{\mathrm{ref}})^2, \qquad \hat{F}_q = -\frac{\partial U_\theta}{\partial R_q}. \tag{146}$$

Here w_F is a fixed loss weight and F_q^{ref} a reference component. Differentiating with respect to a parameter θ_a gives

$$\frac{\partial \mathcal{L}_F}{\partial \theta_a} = -\frac{w_F}{M}\sum_{q=1}^{M}(\hat{F}_q - F_q^{\mathrm{ref}})\frac{\partial^2 U_\theta}{\partial \theta_a \partial R_q}. \tag{147}$$

This example explains why a force-training graph must preserve the coordinate-gradient calculation for further differentiation. It is a generic loss illustration; PET's reported training normalization is given in Section 2.6.1.

For a coordinate Hessian, the second derivative is instead $\partial^2 U_\theta/\partial R_p \partial R_q$, with parameters fixed. An AD system can differentiate a gradient program if the participating operations supply the required higher derivatives. Smoothness and any stopped derivatives or discrete neighbor-selection decisions therefore remain properties of the implemented energy graph. This project uses pretrained parameters; the derivative calculations during harmonic analysis evaluate coordinate curvature rather than optimizer updates. Appendix B explains how many such directional second derivatives are combined to recover the Hessian efficiently.

References

[1] C. Cohen-Tannoudji, B. Diu, and F. Laloë, *Quantum Mechanics, Volume I: Basic Concepts, Tools, and Applications*, 2nd ed., Wiley-VCH, Weinheim (2020), publisher edition.

[2] M. Born and R. Oppenheimer, *Zur Quantentheorie der Molekeln*, Annalen der Physik **84**, 457–484 (1927), doi:10.1002/andp.19270840804.

[3] M. P. Allen and D. J. Tildesley, *Computer Simulation of Liquids*, 2nd ed., Oxford University Press (2017), publisher edition.

[4] M. E. Tuckerman, *Statistical Mechanics: Theory and Molecular Simulation*, 2nd ed., Oxford University Press (2023), publisher edition.

[5] R. M. Martin, *Electronic Structure: Basic Theory and Practical Methods*, 2nd ed., Cambridge University Press (2004), doi:10.1017/CBO9780511805769.

[6] P. Hohenberg and W. Kohn, *Inhomogeneous Electron Gas*, Physical Review **136**, B864–B871 (1964), doi:10.1103/PhysRev.136.B864.

[7] W. Kohn and L. J. Sham, *Self-Consistent Equations Including Exchange and Correlation Effects*, Physical Review **140**, A1133–A1138 (1965), doi:10.1103/PhysRev.140.A1133.

[8] N. Hanikel *et al.*, *Evolution of Water Structures in Metal-Organic Frameworks for Improved Atmospheric Water Harvesting*, Science **374**, 454–459 (2021), doi:10.1126/science.abj0890.

[9] C. L. Hobday *et al.*, *Understanding the Adsorption Process in ZIF-8 Using High Pressure Crystallography and Computational Modelling*, Nature Communications **9**, 1429 (2018), doi:10.1038/s41467-018-03878-6.

[10] C. Rowley, *Periodic Boundary Conditions in 2D* (2015), Wikimedia Commons, CC BY-SA 4.0.

[11] LAMMPS documentation, *compute pressure command*, online documentation (accessed 2 October 2026).

[12] LAMMPS documentation, *fix npt command*, online documentation (accessed 13 October 2026).

[13] LAMMPS developers, *Nosé–Hoover integrator implementation*, `FixNH::compute_scalar`, source code (accessed 15 October 2026).

[14] LAMMPS documentation, *min_style fire command*, online documentation (accessed 13 October 2026).

[15] H. Flyvbjerg and H. G. Petersen, *Error Estimates on Averages of Correlated Data*, Journal of Chemical Physics **91**, 461–466 (1989), doi:10.1063/1.457480.

[16] M. Ceriotti, M. Parrinello, T. E. Markland, and D. E. Manolopoulos, *Efficient Stochastic Thermostatting of Path Integral Molecular Dynamics*, Journal of Chemical Physics **133**, 124104 (2010), doi:10.1063/1.3489925.

[17] R. Diestel, *Graph Theory*, 6th ed., Graduate Texts in Mathematics **173**, Springer (2025), author's textbook site.

[18] A. H. Gebremedhin, A. Pothen, A. Tarafdar, and A. Walther, *Efficient Computation of Sparse Hessians Using Coloring and Automatic Differentiation*, INFORMS Journal on Computing **21**, 209–223 (2009), doi:10.1287/ijoc.1080.0286.

[19] A. Hill, *Graph Coloring*, ASDEX documentation, online documentation.

[20] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*, arXiv:2104.13478 (2021), arXiv:2104.13478.

[21] V. Kapil *et al.*, *Modeling the Structural and Thermal Properties of Loaded Metal–Organic Frameworks: An Interplay of Quantum and Anharmonic Fluctuations*, Journal of Chemical Theory and Computation **15**, 3237–3249 (2019), doi:10.1021/acs.jctc.8b01297.

[22] A. Vaswani *et al.*, *Attention Is All You Need*, Advances in Neural Information Processing Systems **30** (2017), arXiv:1706.03762.

[23] T. B. Brown *et al.*, *Language Models are Few-Shot Learners*, Advances in Neural Information Processing Systems **33**, 1877–1901 (2020), arXiv:2005.14165.

[24] A. Mazitov *et al.*, *PET-MAD as a Lightweight Universal Interatomic Potential for Advanced Materials Modeling*, Nature Communications **16**, 10653 (2025), doi:10.1038/s41467-025-65662-7.

[25] S. N. Pozdnyakov and M. Ceriotti, *Smooth, Exact Rotational Symmetrization for Deep Learning on Point Clouds*, Advances in Neural Information Processing Systems **36** (2023), conference paper; the supplied article is arXiv:2305.19302v3 (6 February 2024), including Appendices A–H.

[26] L. Vanduyfhuys *et al.*, *QuickFF: A Program for a Quick and Easy Derivation of Force Fields for Metal–Organic Frameworks from Ab Initio Input*, Journal of Computational Chemistry **36**, 1015–1027 (2015).

[27] G. J. Martyna, D. J. Tobias, and M. L. Klein, *Constant Pressure Molecular Dynamics Algorithms*, Journal of Chemical Physics **101**, 4177–4189 (1994).

[28] G. J. Martyna, M. L. Klein, and M. Tuckerman, *Nosé–Hoover Chains: The Canonical Ensemble via Continuous Dynamics*, Journal of Chemical Physics **97**, 2635–2643 (1992).

[29] F. Bigi, S. Chong, M. Ceriotti, and F. Grasselli, *A prediction rigidity formalism for low-cost uncertainties in trained neural networks*, Machine Learning: Science and Technology **5**, 045018 (2024), doi:10.1088/2632-2153/ad805f.

[30] M. Kellner and M. Ceriotti, *Uncertainty Quantification by Direct Propagation of Shallow Ensembles*, Machine Learning: Science and Technology **5**, 035006 (2024), doi:10.1088/2632-2153/ad594a.

[31] G. Imbalzano *et al.*, *Uncertainty estimation for molecular dynamics and sampling*, Journal of Chemical Physics **154**, 074102 (2021), doi:10.1063/5.0036522.

[32] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, *Automatic Differentiation in Machine Learning: A Survey*, Journal of Machine Learning Research **18**, 1–43 (2018), JMLR 18(153).

[33] A. Hill, G. Dalle, and A. Montoison, *An Illustrated Guide to Automatic Sparse Differentiation*, ICLR Blogposts (2025), online article.

[34] M. F. Langer, A. Hill, and M. Ceriotti, *Truncated automatic sparse differentiation for machine learning interatomic potentials*, supplied preprint, local paper (SADMOF.pdf). Sections 4–5 and Appendices A–E and H.